Rockchip RT-Thread USB Developer Guide

ID: RK-KF-YF-106

Release Version: V1.6.0

Release Date: 2026-02-13

Security Level: □Top-Secret □Secret □Internal ■Public

DISCLAIMER

THIS DOCUMENT IS PROVIDED “AS IS”. ROCKCHIP ELECTRONICS CO., LTD.(“ROCKCHIP”)DOES NOT PROVIDE ANY WARRANTY OF ANY KIND, EXPRESSED, IMPLIED OR OTHERWISE, WITH RESPECT TO THE ACCURACY, RELIABILITY, COMPLETENESS,MERCHANTABILITY, FITNESS FOR ANY PARTICULAR PURPOSE OR NON-INFRINGEMENT OF ANY REPRESENTATION, INFORMATION AND CONTENT IN THIS DOCUMENT. THIS DOCUMENT IS FOR REFERENCE ONLY. THIS DOCUMENT MAY BE UPDATED OR CHANGED WITHOUT ANY NOTICE AT ANY TIME DUE TO THE UPGRADES OF THE PRODUCT OR ANY OTHER REASONS.

Trademark Statement

"Rockchip", "瑞芯微", "瑞芯" shall be Rockchip’s registered trademarks and owned by Rockchip. All the other trademarks or registered trademarks mentioned in this document shall be owned by their respective owners.

All rights reserved. ©2026. Rockchip Electronics Co., Ltd.

Beyond the scope of fair use, neither any entity nor individual shall extract, copy, or distribute this document in any form in whole or in part without the written approval of Rockchip.

Rockchip Electronics Co., Ltd.

No.18 Building, A District, No.89, software Boulevard Fuzhou, Fujian,PRC

Website: www.rock-chips.com

Customer service Tel: +86-4007-700-590

Customer service Fax: +86-591-83951833

Customer service e-Mail: fae@rock-chips.com

Preface

Overview

This document introduces the USB support of Rockchip RT-Thread SDK, explains the USB code framework and configuration methods, as well as examples of USB Device and Host usage.

Product Version

Chipset	RT-Thread Version
RK1808, RK2108, RK2206, RK3308, RK3358, RK3568, RK625	RT-Thread 3.1.x and above versions
RK2116, RK2118, RK3506	RT-Thread 4.1.x
RK1820	RT-Thread 5.1.x

Intended Audience

This document (this guide) is mainly intended for:

Technical support engineers

Software development engineers

Revision History

Version	Author	Date	Change Description

Version	Author	Date	Change Description
V1.0.0	Wu Liangfeng	2019-07-23	Initial version
V1.0.1	Chen Mouqun	2020-03-17	Correction of links
V1.0.2	Wu Liangfeng	2020-05-25	Correction of format
V1.1.0	Wang Mingcheng	2021-03-12	Addition of UVC usage examples
V1.2.0	Wu Liangfeng Zheng Jianwei	2024-06-11	Addition of RK2118 RT-Thread 4.1.x instructions Addition of Device WINUSB/CDC_VCOM usage examples Improvement of Device HID/MSC usage examples Improvement of Device ADB usage examples, supporting ADB push/pull/shell functions Addition of Device controller's endpoint hardware FIFO configuration instructions Addition of Device VBUS Detect hardware and software usage instructions Addition of Device speed configuration instructions Addition of Host MSC usage examples Improvement of Host configuration instructions
V1.3.0	Wu Liangfeng	2024-09-23	Addition of RT-Thread 4.1.x Device UAC composite device usage method Addition of RT-Thread 4.1.x Multi Host configuration method Addition of RT-Thread 4.1.x RK3506 support instructions
V1.4.0	Wu Liangfeng	2025-08-01	Restructuring of chapters, improvement of USB configuration instructions, enhancement of readability Addition of HAL USB code path and function interface instructions Addition of RT-Thread 5.1.x USB protocol stack instructions Addition of RK1820 support instructions Addition of RK2116 support instructions
V1.5.0	Wu Liangfeng	2025-09-17	Improve the RT-Thread 4.1.x Device UAC usage example. Improve the RT-Thread 5.1.x Host Udisk usage example.

Version	Author	Date	Change Description
V1.6.0	Wu Liangfeng	2026-02-13	Improve RT-Thread 3.1.x/4.1.x Device UAC1 usage examples; Add support for RT-Thread 3.1.x/4.1.x Device UAC2 usage examples; Add support for RT-Thread 5.1.x Device CDC_VCOM usage examples; Add support for RT-Thread 5.1.x Device WINUSB&CDC_ACM usage examples.

Contents

Rockchip RT-Thread USB Developer Guide

1. Rockchip RT-Thread USB Introduction
2. USB Code Framework Introduction
3. USB Code Path
   - 3.1 HAL USB Code Path
     - 3.1.1 HAL USB2.0 OTG Code Path
     - 3.1.2 HAL USB2.0 OTG Function Interface Description
     - 3.1.3 HAL USB2.0 Host Code Path
     - 3.1.4 HAL USB2.0 Host Function Interface Description
     - 3.1.5 HAL USB2.0/3.0 DRD Code Path
     - 3.1.6 HAL USB2.0/3.0 DRD Function Interface Description
   - 3.2 RT-Thread USB Component Code Path
     - 3.2.1 RT-Thread 3.1.x/4.1.x USB Component Code Path
     - 3.2.2 RT-Thread 5.1.x USB Component Code Path
   - 3.3 RT-Thread USB Adaptation Layer Code Path
     - 3.3.1 RT-Thread 3.1.x USB Driver Adaptation Layer Code Path
     - 3.3.2 Code Path for RT-Thread 4.1.x USB Driver Adaptation Layer
     - 3.3.3 Code Path of RT-Thread 5.1.x USB Adaptation Layer
4. USB Configuration
   - 4.1 USB Device Configuration
     - 4.1.1 BSP USB Device Configuration
       - 4.1.1.1 USB Device VBUS Detect Usage Guide
       - 4.1.1.2 USB Device Speed Configuration Guide
       - 4.1.1.3 USB Device Endpoint FIFO Configuration Guide
     - 4.1.2 RT-Thread 3.1.x/4.1.x USB Device Configuration
     - 4.1.3 RT-Thread 5.1.x USB Device Configuration
   - 4.2 USB Host Configuration
     - 4.2.1 BSP USB Host Configuration
       - 4.2.1.1 USB Host VBUS Control Description
     - 4.2.2 RT-Thread 3.1.x/4.1.x USB Host Configuration
     - 4.2.3 RT-Thread 5.1.x USB Host Configuration
5. USB Device Management Interface
6. USB Memory Management Interface
7. USB Device Usage Example
   - 7.1 RT-Thread 3.1.x/4.1.x USB Device Usage Example
     - 7.1.1 USB Device Creation and Registration
     - 7.1.2 USB Device HID Usage Example
     - 7.1.3 USB Device ADB Usage Example
     - 7.1.4 USB Device MSC Usage Example
     - 7.1.5 USB Device UAC1 Usage Example
     - 7.1.6 USB Device UAC2 Usage Example
     - 7.1.7 USB Device UVC Usage Example
     - 7.1.8 USB Device WINUSB Usage Example
     - 7.1.9 USB Device CDC_VCOM Usage Example
   - 7.2 Example of Using RT-Thread 5.1.x USB Device
     - 7.2.1 USB Device ADB Usage Example
     - 7.2.2 USB Device WINUSB Usage Example
     - 7.2.3 USB Device CDC_ACM Usage Example
     - 7.2.4 USB Device CDC_VCOM Usage Example
       - 7.2.4.1 USB Virtual Console Switching Methods
     - 7.2.5 USB Device WINUSB&CDC_ACM Composite Device Usage Example
8. USB Host Usage Example
   - 8.1 Example of Using RT-Thread 3.1.x/4.1.x USB Host

8.1.1 USB Host Creation and Registration
8.1.2 USB Host MSC Usage Example
8.2 Example of Using RT-Thread 5.1.x USB Host
8.2.1 USB Host MSC Usage Example

1. Rockchip RT-Thread USB Introduction

- Supports USB component functions for RT-Thread 3.1.x and above
- The RT-Thread USB component implements the USB Device/Host protocol stack and various USB device class drivers
- RT-Thread 5.1.x version supports both official RTT USB components and CherryUSB components
- All USB controllers use DMA transfer mode

Table 1 Rockchip RT-Thread USB Support Status

Chipset	Supported USB Interfaces	Supported RTT Versions	Supported USB Components
RK1808	1 * USB2.0 HOST	RTT 3.1.x/4.1.x	RTT USB Component
RK1820	1 * USB3.0 DRD + 1 * USB2.0 DRD	RTT 5.1.x	CherryUSB Component
RK2108	1 * USB2.0 Device	RTT 3.1.x/4.1.x	RTT USB Component
RK2116	1 * USB2.0 OTG	RTT 4.1.x	RTT USB Component
RK2118	1 * USB2.0 OTG	RTT 4.1.x	RTT USB Component
RK2206	1 * USB2.0 OTG	RTT 3.1.x	RTT USB Component

Chipset	Supported USB Interfaces	Supported RTT Versions	Supported USB Components
RK3308	1 * USB2.0 OTG + 1 * USB2.0 HOST	RTT 3.1.x/4.1.x	RTT USB Component
RK3358	1 * USB2.0 OTG + 1 * USB2.0 HOST	RTT 3.1.x/4.1.x	RTT USB Component
RK3506	2 * USB2.0 OTG	RTT 4.1.x	RTT USB Component
RK3568	2 * USB2.0 HOST	RTT 3.1.x/4.1.x	RTT USB Component
RK625	1 * USB2.0 OTG	RTT 3.1.x	RTT USB Component

Note

1. The Device/Host mode switching for USB2.0 OTG and USB3.0 DRD interfaces is currently only supported through static configuration in menuconfig;
2. For detailed features of the USB interfaces, please refer to the Rockchip SDK documentation "Rockchip_Developer_Guide_USB_CN".

2. USB Code Framework Introduction

The USB framework is implemented based on the I/O Device Framework provided by RT-Thread, as shown in Figure 1 below, from top to bottom are the I/O Device Management Layer, RTT USB Component, RTT and HAL Adaptation Layer, and HAL Layer USB Controller Driver.

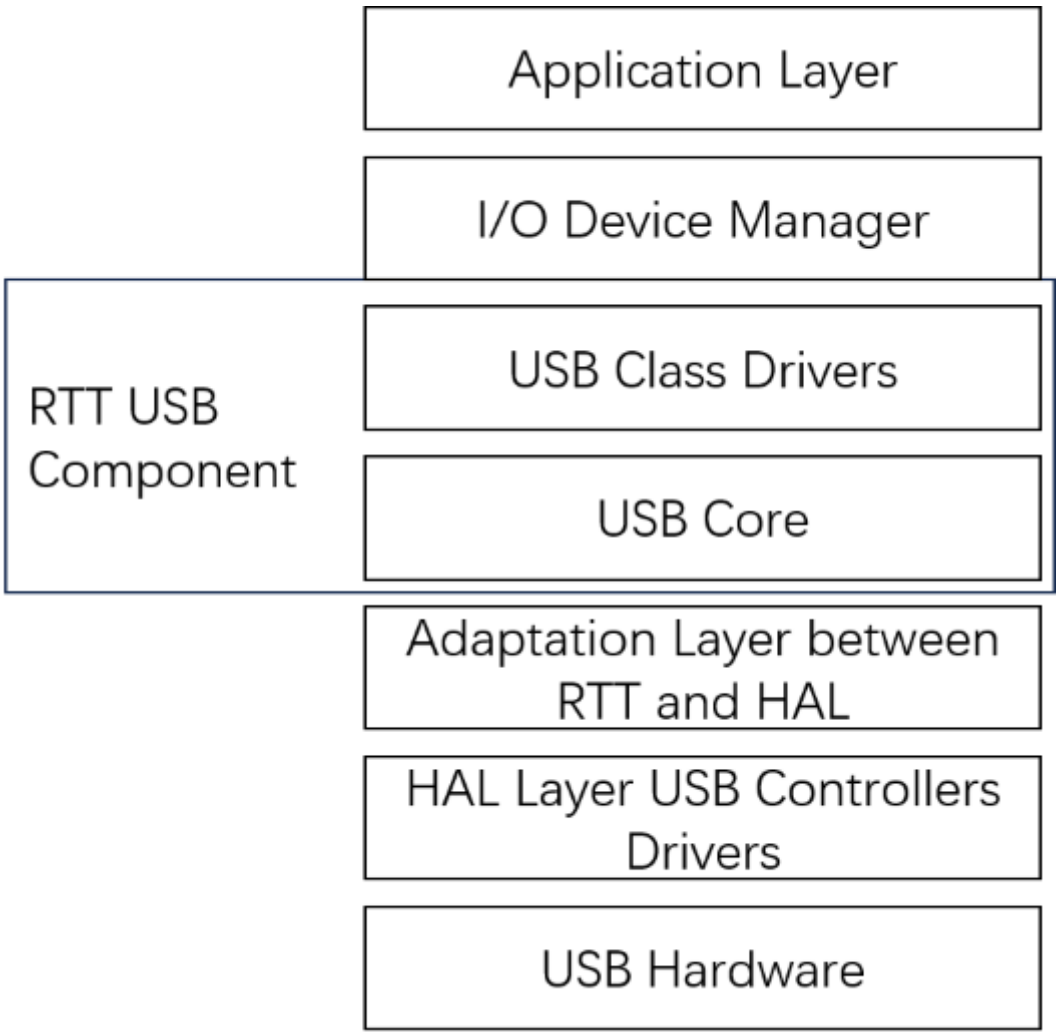


Figure 1 RT-Thread USB Framework Diagram

- Applications access USB device class drivers through I/O device management interfaces, and then interact with the underlying USB controller for data (or control) exchange through the USB device class driver.
- The I/O Device Management Layer implements the encapsulation of USB device class drivers, providing standard interfaces to applications.
- The RTT USB Component implements the USB Device/Host protocol stack and various USB device class drivers. USB device class drivers interact with the underlying USB controller through standard interfaces provided by the USB protocol stack (USB Core). The USB protocol stack interacts with the USB controller through standard interfaces provided by the adaptation layer for data (or control) exchange.
- The RTT and HAL Adaptation Layer is responsible for configuring the hardware information of the Rockchip USB controller and initializing the USB controller through the interfaces provided by the HAL layer USB controller driver. At the same time, according to the standard interfaces defined by the RTT USB Component, it abstracts the HAL layer USB controller driver into interface functions that are convenient for the RTT USB Component to access directly.
- The HAL layer USB controller driver implements the underlying code for reading and writing operations on the USB controller, which can be adapted to different RTOS.

3. USB Code Path

3.1 HAL USB Code Path

3.1.1 HAL USB2.0 OTG Code Path

```
lib/hal/
├── inc
│   ├── hal_hcd.h      /* USB2.0 OTG Host Driver Header File */
│   ├── hal_pcd_ex.h   /* USB2.0 OTG Device Extended Feature Header File */
│   ├── hal_pcd.h      /* USB2.0 OTG Device Driver Header File */
│   ├── hal_usb_core.h /* USB2.0 OTG Core Driver Header File */
│   └── hal_usb_phy.h  /* USB2.0 PHY Driver Header File */
└── src
    ├── hal_hcd.c      /* USB2.0 OTG Host Driver */
    ├── hal_pcd.c      /* USB2.0 OTG Device Driver */
    ├── hal_pcd_ex.c   /* USB2.0 OTG Device Extended Feature Driver */
    ├── hal_usb_core.c /* USB2.0 OTG Core Driver */
    └── hal_usb_phy.c  /* USB2.0 PHY Driver */
```

3.1.2 HAL USB2.0 OTG Function Interface Description

The function interfaces and usage methods of the HAL USB2.0 OTG module can be documented using doxygen and refer to the corresponding HCD/PCD/PCDEx/USB_CORE/USB_PHY modules.

Taking the RK2118 HAL USB2.0 OTG module as an example, the command to generate the documentation with doxygen is as follows:

`hal$ cat tools/Doxyfile tools/doxyfile_rk2118 | doxygen -`

The generated documentation is located at `hal/html/index.html`

As an example of using the HCD HAL module driver, refer to the following:

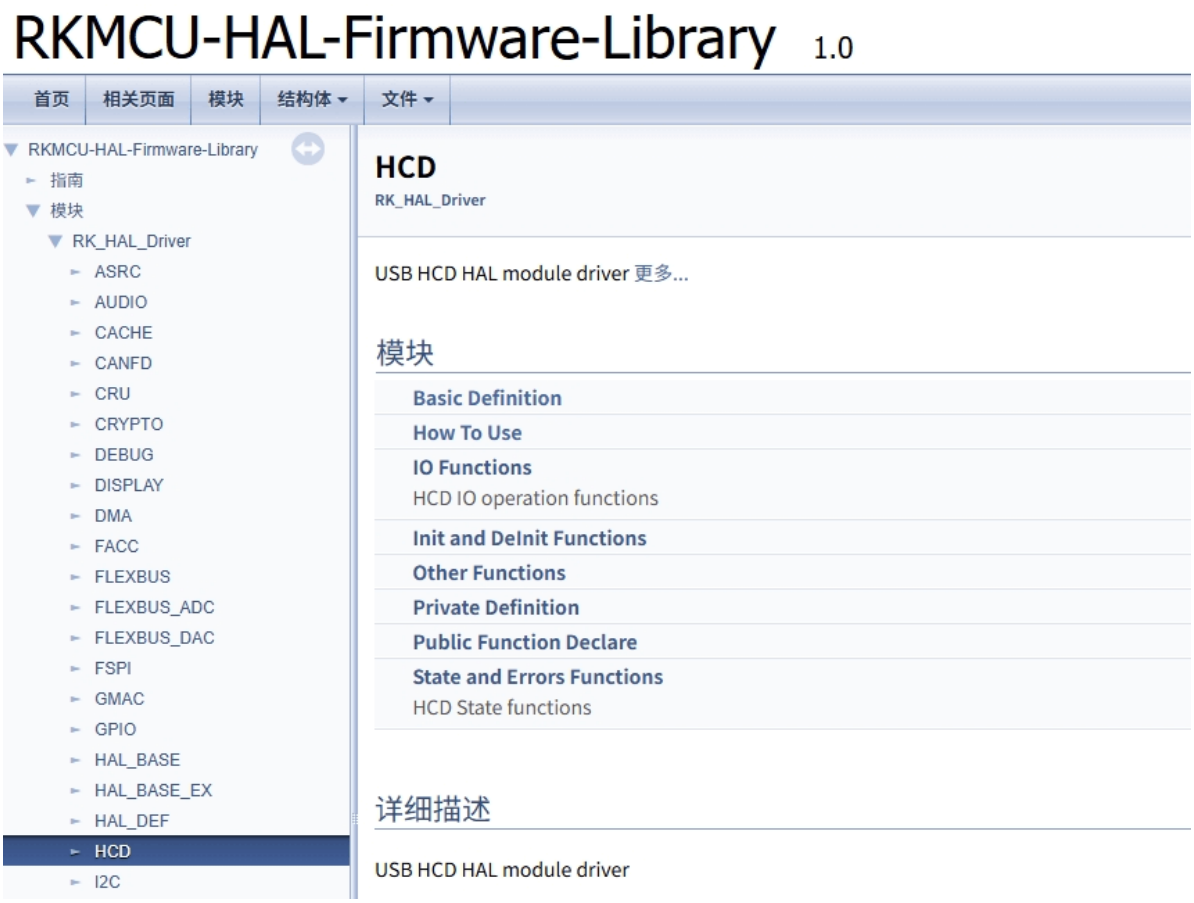


Figure 2 Example of HAL USB2.0 OTG HCD Module Usage

3.1.3 HAL USB2.0 Host Code Path

```
lib/hal/
├── inc
│   ├── hal_ehci.h    /* USB2.0 Host EHCI Controller Driver Header File */
│   ├── hal_ohci.h    /* USB1.1&1.0 Host OHCI Controller Driver Header File */
│   ├── hal_usbh.h    /* USB2.0 Host Core Driver Header File */
│   └── hal_usb_phy.h /* USB2.0 PHY Driver Header File */
└── src
    ├── hal_ehci.c     /* USB2.0 Host EHCI Controller Driver */
    ├── hal_ehci_iso.c /* USB2.0 Host EHCI Controller Isochronous Transfer Driver
*/
    ├── hal_ohci.c     /* USB1.1&1.0 Host OHCI Controller Driver */
    ├── hal_usbh.c     /* USB2.0 Host Core Driver */
    └── hal_usb_phy.c  /* USB2.0 PHY Driver */
```

3.1.4 HAL USB2.0 Host Function Interface Description

The function interfaces and usage methods of the HAL USB2.0 Host module can be documented using doxygen and refer to the corresponding EHCI/OHCI/USB_HOST/USB_PHY module sections.

Taking the RK3568 HAL USB2.0 Host module as an example, the command to generate documentation with doxygen is as follows:

```
hal$ cat tools/Doxyfile tools/doxyfile_rk3568 | doxygen -
```

The generated documentation is located at `hal/html/index.html`

As an example of using the EHCI HAL module driver, refer to the following:



Figure 3 Example of HAL USB2.0 EHCI Module Usage

3.1.5 HAL USB2.0/3.0 DRD Code Path

```
lib/hal/
├── inc
│   ├── hal_drd_core.h   /* USB DRD Core Driver Header File */
│   ├── hal_drd_gadget.h /* USB DRD Gadget Driver Header File */
│   ├── hal_xhci.h       /* USB DRD Host xHCI Driver Header File */
│   └── hal_usb_phy.h    /* USB2.0&3.0 PHY Driver Header File */
└── src
    ├── hal_drd_core.c   /* USB DRD Core Driver */
    ├── hal_drd_gadget.c /* USB DRD Gadget Driver */
    ├── hal_usb_phy.c    /* USB2.0&3.0 PHY Driver */
    └── hal_xhci.c       /* USB DRD Host xHCI Driver */
```

3.1.6 HAL USB2.0/3.0 DRD Function Interface Description

The function interfaces and usage methods of the HAL USB2.0/3.0 DRD module can be documented using Doxygen and reviewed in corresponding sections such as USB_DRD_CORE/USB_DRD_GADGET/USB_PHY/USB_XHCI.

Taking the RK1820 HAL USB DRD module as an example, the command to generate documentation with Doxygen is as follows:

```
hal$ cat tools/Doxyfile tools/doxyfile_rk1820 | doxygen -
```

The generated documentation is located at `hal/html/index.html`

As an example of using the HAL USB_DRD_GADGET module driver, refer to the following:

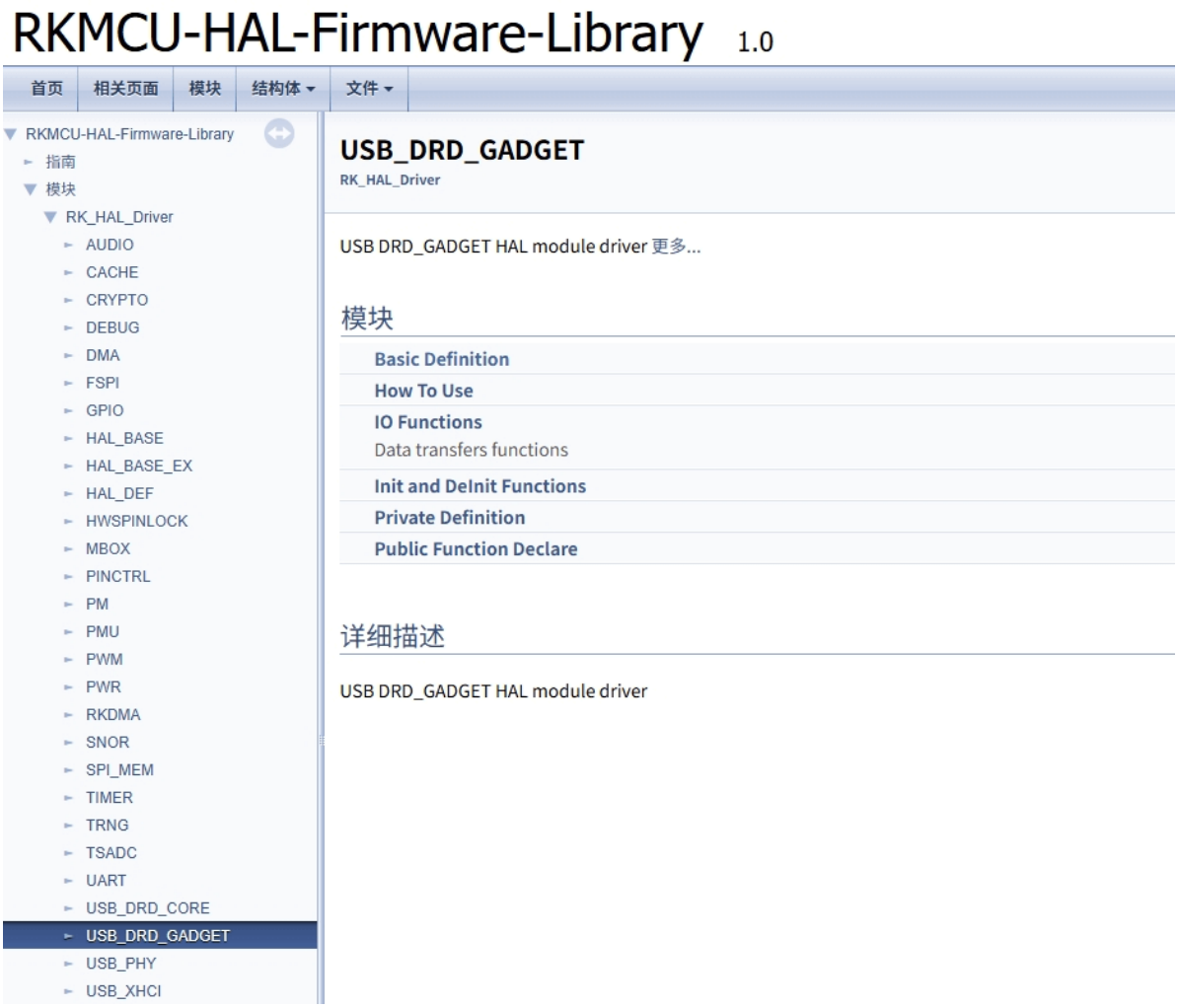


Figure 4 Example of HAL USB_DRD_GADGET Module Usage

3.2 RT-Thread USB Component Code Path

3.2.1 RT-Thread 3.1.x/4.1.x USB Component Code Path

RT-Thread USB Component is located at `rt-thread/components/drivers/usb`, which is a high-level abstraction of the USB protocol stack logic based on the USB2.0 protocol specification, supporting both Host (host) and Device (slave) modes. This component provides a friendly porting interface for driver migration, allowing developers to directly integrate the HCD (Host Controller Driver) and PCD (Peripheral Controller Driver) driver code from the manufacturer's BSP into RT-Thread.

The framework code path of the USB component is as follows:

```
rt-thread$ tree components/drivers/usb
components/drivers/usb
├── SConscript
├── usbdevice
│   ├── class  /* USB Device Class Drivers */
│   │   ├── audio_mic.c
│   │   ├── audio_speaker.c
│   │   ├── cdc.h
```

```
│   │   ├── cdc_vcom.c
│   │   ├── ecm.c
│   │   ├── hid.c
│   │   ├── hid.h
│   │   ├── mstorage.c
│   │   ├── mstorage.h
│   │   ├── ndis.h
│   │   ├── rndis.c
│   │   ├── rndis.h
│   │   ├── uac1.c
│   │   ├── uac1.h
│   │   ├── uaudioreg.h
│   │   ├── winusb.c
│   │   └── winusb.h
│   ├── core  /* USB Device Core Code, Implementing Device Protocol Stack */
│   │   ├── usbdevice.c
│   │   └── usbdevice_core.c
│   └── SConscript
└── usbhost
    ├── class /* USB Host Class Drivers */
    │   ├── adk.c
    │   ├── adk.h
    │   ├── hid.c
    │   ├── hid.h
    │   ├── mass.c
    │   ├── mass.h
    │   ├── udisk.c
    │   ├── ukbd.c
    │   └── umouse.c
    ├── core  /* USB Host Core Code, Implementing Host Protocol Stack */
    │   ├── driver.c
    │   ├── hub.c
    │   ├── usbhost.c
    │   └── usbhost_core.c
    └── SConscript
```

Note

- RT-Thread 3.1.x/4.1.x USB Component USB Device Class Support Status

1. CDC-ACM (Communication Device Class - Abstract Control Model, USB Virtual Serial Port)
2. CDC-ECM (Communication Device Class - Ethernet Networking Control Model, USB Ethernet Control Model)
3. HID (Human Interface Device, Ergonomic Input Device, Supporting Keyboard/Mouse/General HID/Media Keyboard)
4. MStorage (Mass Storage, Mass Storage Device)
5. RNDIS (Remote Network Driver Interface Specification, Remote Network Driver Interface)
6. WINUSB (Windows USB, Microsoft Universal USB)

- RT-Thread 3.1.x/4.1.x USB Component USB Host Class Support Status

1. MStorage (Mass Storage, Mass Storage Device)
2. HUB (Basic Driver Supported, Functionality Under Debugging)
3. HID (Basic Driver Supported, Functionality Under Debugging)

3.2.2 RT-Thread 5.1.x USB Component Code Path

RT-Thread 5.1.x supports two types of USB components:

1. RTT Legacy USB component, which is the same as the RT-Thread 3.1.x/4.1.x USB component.
2. CherryUSB component

```
rt-thread$ tree -d components/drivers/usb/cherryusb
components/drivers/usb/cherryusb
├── class   /* USB class master-slave drivers */
│   ├── adb
│   ├── aoa
│   ├── audio
│   ├── cdc
│   ├── dfu
│   ├── hid
│   ├── hub
│   ├── midi
│   ├── msc
│   ├── template
│   ├── vendor
│   │   ├── net
│   │   ├── serial
│   │   ├── wifi
│   │   └── xbox
│   ├── video
│   └── wireless
├── common  /* USB specification definitions, common macros, standard interface
definitions */
├── core    /* Core implementation of USB master-slave protocol stack */
├── demo    /* Master-slave class demos */
│   ├── adb
│   └── bootuf2
├── osal    /* OS encapsulation layer */
└── port    /* Porting interfaces required for USB master-slave */
    ├── aic
    ├── bouffalolab
    ├── ch32
    ├── chipidea
    ├── dwc2
    ├── ehci
    ├── fsdev
    ├── hpm
    ├── kinetis
    ├── musb
    ├── nrf5x
    ├── ohci
    ├── pusb2
    │   └── rt-thread
    ├── rp2040
    ├── template
    └── xhci
        └── phytium
```

```
            └── rt-thread

/* CherryUSB documentation */
rt-thread$ components/drivers/usb/cherryusb/README_en.md

/* CherryUSB board-level USB config file, using RK1820 SDK as an example */
rt-thread$ bsp/rockchip/rk1820/board/common/usb_config.h
```

Note

1. RT-Thread 5.1.x recommends using the CherryUSB component as a priority for USB function development;
2. CherryUSB source code download: https://github.com/cherry-embedded/CherryUSB
3. CherryUSB usage guide: https://cherryusb.readthedocs.io/en/latest/index.html
4. Rockchip SDK USB ADB driver does not use the class ADB driver in the CherryUSB component, but instead uses the ADB driver developed by Rockchip itself, with the source code located at:

   `rt-thread/third_party/adbd`
5. Rockchip SDK does not use the port interface driver in the CherryUSB component, but instead uses the HAL repository USB driver.

3.3 RT-Thread USB Adaptation Layer Code Path

3.3.1 RT-Thread 3.1.x USB Driver Adaptation Layer Code Path

```
rt-thread$ tree bsp/rockchip/common/drivers/
bsp/rockchip/common/drivers/
├── drv_usbd.c     /* Adaptation layer between RT-Thread and HAL USB2.0 OTG
Device controller driver */
├── drv_usbd.h     /* Header file for the adaptation layer between RT-Thread and
HAL USB2.0 OTG Device controller driver */
├── drv_usbhost.c  /* Adaptation layer between RT-Thread and HAL USB2.0 Host
EHCI&OHCI controller driver */
└── drv_usbh.c     /* Adaptation layer between RT-Thread and HAL USB2.0 OTG Host
controller driver */
```

3.3.2 Code Path for RT-Thread 4.1.x USB Driver Adaptation Layer

```
rt-thread$ tree bsp/rockchip/common/drivers/
bsp/rockchip/common/drivers/
├── drv_usbd.c     /* Adaptation layer between RT-Thread and HAL USB2.0 OTG
Device controller driver */
├── drv_usbd.h     /* Header file for the adaptation layer between RT-Thread and
HAL USB2.0 OTG Device controller driver */
├── drv_usbhost.c  /* Adaptation layer between RT-Thread and HAL USB2.0 Host
EHCI&OHCI controller driver */
└── drv_usbotgh.c  /* Adaptation layer between RT-Thread and HAL USB2.0 OTG Host
controller driver */
```

3.3.3 Code Path of RT-Thread 5.1.x USB Adaptation Layer

```
rt-thread$ tree bsp/rockchip/common/drivers
bsp/rockchip/common/drivers
├── cherryusb
│   ├── cherryusb_dc_drd.c  /* Adaptation layer between cherryusb and HAL USB DRD
Gadget controller driver */
│   ├── cherryusb_hc_xhci.c /* Adaptation layer between cherryusb and HAL USB DRD
xHCI controller driver */
│   ├── cherryusb_mem.c     /* Memory management provided by cherryusb to HAL USB
driver */
│   └── SConscript
├── drv_usbd.c    /* Adaptation layer between RTT Legacy USB component and HAL
USB2.0 OTG Device controller driver */
├── drv_usbd.h    /* Header file for adaptation layer between RTT Legacy USB
component and HAL USB2.0 OTG Device controller driver */
├── drv_usbhost.c /* Adaptation layer between RTT Legacy USB component and HAL
USB2.0 Host EHCI&OHCI controller driver */
└── drv_usbotgh   /* Adaptation layer between RTT Legacy USB component and HAL
USB2.0 OTG Host controller driver */
```

4. USB Configuration

4.1 USB Device Configuration

4.1.1 BSP USB Device Configuration

USB2.0 OTG Device BSP reference configuration is as follows:

```
#ifdef HAL_PCD_MODULE_ENABLED
const struct HAL_USB_DEV g_usbdDev =
{
    .pReg = USB_OTG,
    .hclkGateID = HCLK_USBOTG_GATE,
    .utmiclkGateID = PCLK_USBPHY_GATE,
    .irqNum = OTG_IRQn,
```

```
    .BvalidIrqNum = OTG_BVALID_IRQn,
    .cfg =
    {
        .epNum = 10,  /* Total number of supported USB IN/OUT endpoints */
        .ep0Mps = USB_OTG_MAX_EP0_SIZE,
        .phyif = USB_PHY_UTMI_WIDTH_16,
        .speed = USB_OTG_SPEED_HIGH,
        .dmaEnable = true, /* Enable USB DMA transfer */
        .sofEnable = false,
        .lpmEnable = false,
        .vbusSensingEnable = false,
        .suspendEnable = true, /* Enable USB Autosuspend feature */
    },
};
#endif
```

USB3.0 DRD Device BSP reference configuration is as follows:

```
#ifdef HAL_USB_DRD_MODULE_ENABLED
const struct HAL_USBDRD_DEV g_usbdrdDev =
{
    .pReg = USB3DRD,
    .irqNum = USB3OTG_IRQn,
    .busGateID = ACLK_USB3OTG0_GATE,
    .refGateID = CLK_REF_USB3OTG0_GATE,
    .suspendGateID = CLK_SUSPEND_USB3OTG0_GATE,
    .cfg =
    {
        .drMode = USB_DR_MODE_PERIPHERAL, /* Configure DRD device mode */
        .hsphyMode = USBPHY_INTERFACE_MODE_UTMIW,
        .maximumSpeed = DRD_USB_SPEED_SUPER, /* Configure the maximum supported
transfer rate based on chip design and actual product */
        .hsphyId = 0, /* Configure USB2.0 PHY ID based on chip design */
        .ssphyId = 1, /* Configure USB3.0 PHY ID based on chip design */
        .hasAwidth64 = 1, /* Enable DMA access to 64-bit bus addresses */
        .disDelPhyPowerChgQuirk = 1,
        .disEnblslpmQuirk = 1,
        .disRxdetInp3Quirk = 1,
        .disTxIpGapLinecheckQuirk = 1,
        .disU1u2Quirk = 1,
        .disU2SusphyQuirk = 1,
        .disU2FreeclkExistsQuirk = 1,
        .disU3SusphyQuirk = 1,
        .parkmodeDisableHsQuirk = 1,
        .parkmodeDisableSsQuirk = 1,
        .u2lpmDisable = 1,
    },
};
#endif
```

4.1.1.1 USB Device VBUS Detect Usage Guide

Applicable Platforms: RK2108/RK2116/RK2118/RK1820

Supports three different VBUS Detect methods:

- GPIO VBUS Detect: Detect VBUS level through GPIO

  Corresponding macro definition `USB_VBUS_PIN`

  Reference RK2118 evb hardware and software design: `rt-thread/bsp/rockchip/rk2118/board/evb/board.h`
- PMIC VBUS Detect: Detect VBUS level through PMIC

  Corresponding macro definition `RT_USING_USB_PMIC_VBUS`

  Reference RK2108 recording_pen hardware and software design: `rt-thread/bsp/rockchip/rk2108/board/recording_pen_v10/board.h`
- USB PHY VBUS Detect: Detect VBUS level through USB PHY

  Corresponding macro definition `RT_USING_BVALID_IRQ`

  Reference RK2118 iotest hardware and software design: `rt-thread/bsp/rockchip/rk2118/board/iotest/board.h`

4.1.1.2 USB Device Speed Configuration Guide

Applicable Platforms: RK2108/RK2116/RK2118

USB Device supports High speed by default. If the product requires the device to operate at Full speed (e.g., UAC product form), it can be configured by defining `RT_USING_USBD_SPEED_FULL`.

4.1.1.3 USB Device Endpoint FIFO Configuration Guide

Applicable Platforms: RK2108/RK2116/RK2118

USB Device Endpoint FIFO refers to the hardware FIFO allocated to each endpoint within the USB controller. All Rx endpoints share one RxFIFO, while each Tx endpoint has its own TxFIFO. The specific implementation is in the USB Device controller driver adaptation layer and board-level configuration.

Taking RK2118 SDK as an example:

The board-level default configuration for USB endpoints is in `rt-thread/bsp/rockchip/rk2118/board/common/board_base.c`, as shown below. The g_usb_ep_pool is set according to the USB functionality of most product forms, which can support 2 BULK endpoints, 2 ISOC endpoints, and 2 INT endpoints simultaneously. If the product needs to support other USB composite devices, the configuration should be adjusted according to the actual number and type of USB endpoints required.

```
#ifdef RT_USING_USB_DEVICE
RT_WEAK struct ep_id g_usb_ep_pool[] =
{
    { 0x0,  USB_EP_ATTR_CONTROL,    USB_DIR_INOUT,  64,   ID_ASSIGNED   },
    { 0x1,  USB_EP_ATTR_BULK,       USB_DIR_IN,     1024, ID_UNASSIGNED },
    { 0x2,  USB_EP_ATTR_BULK,       USB_DIR_OUT,    512,  ID_UNASSIGNED },
    { 0x3,  USB_EP_ATTR_ISOC,       USB_DIR_IN,     1024, ID_UNASSIGNED },
    { 0x4,  USB_EP_ATTR_ISOC,       USB_DIR_OUT,    512,  ID_UNASSIGNED },
    { 0x5,  USB_EP_ATTR_INT,        USB_DIR_IN,     64,   ID_UNASSIGNED },
    { 0x6,  USB_EP_ATTR_INT,        USB_DIR_OUT,    64,   ID_UNASSIGNED },
    { 0xFF, USB_EP_ATTR_TYPE_MASK,  USB_DIR_MASK,   0,    ID_ASSIGNED   },
};
#endif
```

The FIFO settings for USB HAL are in `rt-thread/bsp/rockchip/common/drivers/drv_usbd.c`

Call `HAL_PCDEx_SetRxFiFo` to set RxFIFO, unit 4 bytes

Call `HAL_PCDEx_SetTxFiFo` to set TxFIFO, unit 4 bytes

```
static void usb_pcd_lowlevel_hw_init(struct PCD_HANDLE *pcd)
{
    uint8_t i;

    /* Initialize Low Level Driver */
    HAL_PCD_Init(pcd);

#ifndef RT_USB_DEVICE_UVC
    HAL_PCDEx_SetRxFiFo(pcd, 0x118);
#else
    /*
     * Set smaller RxFiFo for 3 * UVC interfaces, then
     * it can assign enough TxFiFo size (1024 Bytes)
     * for each UVC streaming IN endpoint to get higher
     * performance.
     */
    HAL_PCDEx_SetRxFiFo(pcd, 0x8C);
#endif

    /* Set TxFIFOs according to the g_usb_ep_pool array */
    for (i = 0; i < pcd->cfg.epNum + 1; i++)
    {
        if (g_usb_ep_pool[i].addr == 0xFF)
            break;

        if (g_usb_ep_pool[i].dir == USB_DIR_IN || g_usb_ep_pool[i].dir ==
USB_DIR_INOUT)
            HAL_PCDEx_SetTxFiFo(pcd, g_usb_ep_pool[i].addr,
                                (g_usb_ep_pool[i].maxpacket >> 2));
    }

    HAL_PCD_Start(pcd);
}
```

4.1.2 RT-Thread 3.1.x/4.1.x USB Device Configuration

USB Device can support a single interface or a combination of multiple interfaces in a composite device (Composite Device), and the USB component allows developers to enable composite functionality through the macro `RT_USB_DEVICE_COMPOSITE`.

```
RT-Thread Components  --->
    Device Drivers  --->
        Using USB  --->
            [ ] Using USB host
            [*] Using USB device
            (4096) usb thread stack size
            (0x2207) USB Vendor ID
            (0x0007) USB Product ID
            [ ]   Enable composite device
                    Device type (Using custom class by register interface)  --->
```

Note

1. `USB Vendor ID` is 0x2207, which is the VID officially authorized by USB-IF to Rockchip and has uniqueness. USB-IF stipulates that VID is unique to each Vendor, and the same VID cannot be authorized to different Vendors. Therefore, the Rockchip USB VID is only for product development and debugging; if the product requires USB-IF certification, the developer must apply for USB-IF membership to obtain an officially authorized unique Vendor ID;
2. `USB Product ID` is 0x0007, which can be defined according to the actual product, with the rule being 0x00XX, keeping the high 8 bits as 0 to avoid conflicts with Maskrom/Loader USB PID;
3. `Device type` can be further selected based on the actual product needs, such as CDC/MSC/HID/WINUSB/UAC1, etc.;
4. `Enable composite device` is used to support USB composite devices, such as the combination of MSC + CDC-ACM. If it is a single USB function, this option does not need to be enabled;
5. The USB Device function is mainly suitable for MCU chips, such as: RK2108/RK2118/RK625.

4.1.3 RT-Thread 5.1.x USB Device Configuration

Based on the product USB Device requirements, enable USB component configuration.

```
RT-Thread Components  --->
    Device Drivers  --->
        [*] Using USB with CherryUSB  --->
            [*]   Enable usb device mode  --->
                  Select usb device speed (AUTO) /* Select USB Speed automatic
detection */
                  Select usb device ip (drd_rk)  /* Select USB Device controller
IP */
                  [ ]   Enable usb cdc acm device /* Select USB class driver */
                  ...
                  [ ]   Enable usb dfu device (NEW)
```

```
                        Select usb device template ---> /* Select USB device demo
driver */
                            ( ) cdc_acm
                            ( ) msc_ram
                            ...
                            ( ) winusbv2_hid
```

Based on the hardware design, enable USB controller driver configuration. Taking RK1820 USB0 DRD Device configuration as an example:

```
RT-Thread Rockchip RK1820 drivers  --->
    Enable USB DRD Device  --->
        [*] Enable USB DRD Device controllers
            [*]   Enable USB DRD_0 Device
```

4.2 USB Host Configuration

4.2.1 BSP USB Host Configuration

USB2.0 OTG Host BSP reference configuration is as follows:

```
#ifdef HAL_HCD_MODULE_ENABLED
const struct HAL_USB_DEV g_usbotgh0Dev =
{
    .pReg = USB_OTG,
    .hclkGateID = HCLK_USBOTG_GATE,
    .utmiclkGateID = PCLK_USBPHY_GATE,
    .irqNum = OTG_IRQn,
    .BvalidIrqNum = OTG_BVALID_IRQn,
    .cfg =
    {
        .phyif = USB_PHY_UTMI_WIDTH_16,
        .speed = USB_OTG_SPEED_HIGH,
        .hcNum = 8, /* Total number of supported USB Host channels */
        .dmaEnable = true,
        .sofEnable = false,
        .lpmEnable = false,
        .suspendEnable = true, /* Enable USB Autosuspend function */
    },
};
#endif
```

USB2.0 DRD Host BSP reference configuration is as follows:

```
#ifdef HAL_USB_XHCI_MODULE_ENABLED
const struct HAL_USBXHCI_DEV g_usbxhciDev =
{
    .pReg = USB2DRD,
    .irqNum = USB2OTG_IRQn,
    .busGateID = ACLK_USB2OTG_GATE,
    .refGateID = CLK_REF_USB2OTG_GATE,
```

```
    .cfg =
    {
        .drMode = USB_DR_MODE_HOST, /* Configure DRD Host mode */
        .hsphyMode = USBPHY_INTERFACE_MODE_UTMIW,
        .maximumSpeed = DRD_USB_SPEED_HIGH, /* Configure the maximum supported
transfer rate based on chip design and actual product */
        .hsphyId = 0, /* Configure USB2.0 PHY ID based on chip design */
        .hasAwidth64 = 1, /* Enable DMA access to 64-bit bus addresses */
        .disDelPhyPowerChgQuirk = 1,
        .disEnblslpmQuirk = 1,
        .disRxdetInp3Quirk = 1,
        .disTxIpGapLinecheckQuirk = 1,
        .disU1u2Quirk = 1,
        .disU2SusphyQuirk = 1,
        .disU2FreeclkExistsQuirk = 1,
        .disU3SusphyQuirk = 1,
        .parkmodeDisableHsQuirk = 1,
        .parkmodeDisableSsQuirk = 1,
        .u2lpmDisable = 1,
    },
};
#endif
```

4.2.1.1 USB Host VBUS Control Description

Applicable Platforms: RK2116/RK2118/RK1820/RK3506/RK3568

Support for controlling the 5V VBUS output through GPIO can be achieved by adding the corresponding GPIO BANK_PIN macro definition for VBUS in the board-level header file.

1. USB2.0 OTG Host VBUS GPIO

   Taking RK3506 EVB1 as an example, the VBUS macro definition is in `bsp/rockchip/rk3506-32/board/evb1/board.h`

   ```
   #define USB_OTG_HOST0_VBUS_PIN        BANK_PIN(GPIO_BANK1, 20)
   #define USB_OTG_HOST1_VBUS_PIN        BANK_PIN(GPIO_BANK1, 24)
   ```

2. USB2.0 Host VBUS GPIO

   Taking RK3568 EVB1 as an example, the VBUS macro definition is in `bsp/rockchip/rk3568-32/board/rk3568_evb1/board.h`

   ```
   #define USB_HOST_VBUS_PIN        BANK_PIN(GPIO_BANK0, 6)
   ```

4.2.2 RT-Thread 3.1.x/4.1.x USB Host Configuration

USB Host currently only supports Mass Storage functionality, with HUB and HID functionalities under development.

Configuration Method for Mass Storage Functionality

```
RT-Thread Components  --->
    Device Drivers  --->
        Using USB  --->
            [*] Using USB host
            [*]   Enable Udisk Drivers
            (/)     Udisk mount dir
            [ ]   Enable HID Drivers
            [ ] Using USB device
            (4096) usb thread stack size
```

USB Host Driver Adaptation Layer Configuration Items, located under the SoC drivers path, with RK2116 USB HOST configuration as an example:

```
RT-Thread rockchip RK2116 drivers  --->
    Enable USB HOST  --->
        [*] Enable USB OTG HOST
        [*]   Enable USB OTG0 HOST
```

Note

1. USB Host functionality is applicable to both MCU and AP chips. The configuration of the host driver adaptation layer needs to be selected based on the actual chip USB controller model;
2. Due to the limitations of the USB driver adaptation layer, it does not support enabling both `Using USB host` and `Using USB device` simultaneously, otherwise, it will cause USB functionality abnormalities.

4.2.3 RT-Thread 5.1.x USB Host Configuration

Based on the product USB Host requirements, enable USB component configuration

```
RT-Thread Components  --->
    Device Drivers  --->
        [*] Using USB with CherryUSB  --->
            [*]   Enable usb host mode  --->
                  Select usb host ip (xhci_rk)    /* Select USB Host Controller
IP xhci_rk */
                  [ ]   Enable usb cdc acm driver /* Select USB class driver */
                  ...
                  [ ]   Enable usb msc driver
                  [ ]   Use usb host template      /* USB Host Test Demo for test
acm/hid/... */
```

Based on the hardware design, enable USB controller driver configuration. Taking the RK1820 USB1 DRD Host configuration as an example:

```
RT-Thread rockchip RK1820 drivers  --->
    Enable USB XHCI HOST  --->
        [*] Enable USB XHCI HOST controllers
        [*]   Enable USB XHCI1 HOST
```

5. USB Device Management Interface

USB applications access USB hardware through the I/O device management interfaces provided by RT-Thread, as shown below:

Table 2 USB Device Management Interfaces

Function	Description
rt_device_find()	Find USB Device
rt_device_open()	Open USB Device
rt_device_read()	Read USB Data
rt_device_write()	Write USB Data
rt_device_control()	Control USB Device

6. USB Memory Management Interface

Rockchip's full series of USB controllers default to using DMA transfers, hence when performing USB read/write operations, it is necessary to ensure that the data Buffer's starting address is aligned according to the cacheline, and to maintain the consistency of DCache.

USB Date Buffer Memory Allocation and Release Interface

It is recommended to prioritize the use of rt_malloc_align/rt_free_align. The commonly used memory management interfaces and their descriptions in RT-Thread are shown in Table 3 below.

Table 3 RT-Thread Memory Management Interface

RTT malloc API	Functional Description	Usage Scenario
rt_malloc rt_free	1. API defined by RTT system; 2. Allocates memory from the system heap; 3. Cannot guarantee memory address alignment; 4. Does not guarantee uncache, when the system's cache is enabled, software-driven cache flushing is required to ensure cache consistency.	Widely used in RTT components and bsp/rockchip/common for allocating non-aligned address and cache-allowed memory.
rt_malloc_align rt_free_align	1. API defined by RTT system; 2. Allocates memory from the system heap; 3. Can guarantee memory address alignment, and the align size is configurable, implemented based on the rt_malloc API; 4. Does not guarantee uncache, when the system's cache is enabled, software-driven cache flushing is required to ensure cache consistency.	Widely used in RTT components and bsp/rockchip/common for allocating aligned address and cache-allowed memory. For example: components/drivers/usb/usbdevice/class/uac1.c components/drivers/usb/usbdevice/class/hid.c bsp/rockchip/common/drivers/drv_usbhost.c
rt_dma_malloc rt_free_align	1. RK's custom API (dependent on RT_USING_DMA common/drivers/dma.c); 2. Allocates DMA memory from the system heap; 3. Can guarantee memory address alignment based on CACHE_LINE_SIZE and DMA_ALIGN_SIZE, implemented based on the rt_malloc_align API; 4. Does not guarantee uncache, when the system's cache is enabled, software-driven cache flushing is required to ensure cache consistency.	Used in bsp/rockchip/common for modules that explicitly use SoC DMA. For example: bsp/rockchip/common/drivers/drv_spinand.c bsp/rockchip/common/drivers/drv_sdio.c bsp/rockchip/common/drivers/drv_pl330.c
rt_malloc_uncache rt_free_uncache	1. RK's custom API (dependent on RT_USING_UNCACHE_HEAP common/drivers/drv_heap.c); 2. Allocates DMA memory from the specified uncache heap; 3. Cannot guarantee memory address alignment, implemented based on the rt_memheap_alloc API; 4. Can guarantee uncache, when the system's cache is enabled, no need for software-driven cache flushing to ensure cache consistency.	1. Used in bsp/rockchip/common for modules that must use uncache memory. For example: bsp/rockchip/common/drivers/drv_gmac.c bsp/rockchip/common/drivers/drv_usbhost.c 2. Requires calling rt_uncache_heap_init to initialize the uncache heap at system startup For example: rk3568-32/board/common/board_base.c
rt_dma_malloc_large rt_dma_free_large	1. RK's custom API (dependent on RT_USING_UNCACHE_HEAP and RT_USING_LARGE_HEAP common/drivers/drv_heap.c); 2. Allocates DMA memory from the specified large heap; 3. Can guarantee memory address alignment based on CACHE_LINE_SIZE and DMA_ALIGN_SIZE, implemented based on the rt_memheap_alloc API; 4. Does not guarantee uncache, when the system's cache is enabled, software-driven cache flushing is required to ensure cache consistency.	1. Used in bsp/rockchip/common for modules that require large DMA memory. For example: drv_display/drv_spi_screen/camera 2. Requires calling rt_large_heap_init to initialize the large heap at system startup, currently only used in a few platforms such as RK2108/RK2206 For example: rk2108/start/startup.c

USB Data Buffer DCache Maintain Interface

1. DCache Maintain Interface Provided by RT-Thread

   `void rt_hw_cpu_dcache_ops(int ops, void *addr, int size);`

   Table 4 DCache Maintenance Interface Parameter Description

Parameter	Description
ops	The maintain operation to be performed, RT_HW_CACHE_FLUSH or RT_HW_CACHE_INVALIDATE
addr	The starting address for the maintain operation,
size	The size of the maintain operation

   Notes on usage: The address range for the maintain operation should cover the modified data but should not exceed it. Here is a specific example:

```
buf = rt_dma_malloc(buf_size);
modify_start = &buf[50];
device_modify_data(modify_start, buf_size-50);

/* Note that the starting address should not use buf, and the size should not
use buf_size, just cover the range of the modified data */
rt_hw_cpu_dcache_ops(RT_HW_CACHE_INVALIDATE, modify_start, buf_size-50);
```

   USB driver usage scenario: `bsp/rockchip/common/drivers/drv_usbhost.c`

2. DCache Maintain Interface Provided by HAL

```
HAL_DCACHE_CleanByRange(uintptr_t address, uint32_t sizeByte);
HAL_DCACHE_InvalidateByRange(uintptr_t address, uint32_t sizeByte);
HAL_DCACHE_CleanInvalidateByRange(uintptr_t address, uint32_t sizeByte);
```

   USB driver usage scenarios:

```
lib/hal/src/hal_hcd.c
lib/hal/src/hal_pcd.c
lib/hal/src/hal_drd_gadget.c
```

7. USB Device Usage Example

7.1 RT-Thread 3.1.x/4.1.x USB Device Usage Example

7.1.1 USB Device Creation and Registration

USB Device is registered into the RT-Thread I/O device manager through the `rt_device_register()` interface.

Here, the USB Device refers to the device registered by the USB controller in the RT-Thread I/O device manager, which is exclusively accessible by USB device class drivers (such as HID/MStorage, etc.). Application programs cannot directly access the USB controller device. Additionally, different USB device class drivers may also create and register corresponding devices in the RT-Thread I/O device manager to facilitate application access to USB devices. For example, `components/drivers/usb/usbdevice/class/hid.c` creates a `hidd` device node through the `rt_device_register` interface.

The creation and registration of the USB controller Device are implemented in the RTT and HAL adaptation layer `bsp/rockchip/common/drivers/drv_usbd.c`, and the process is shown in the figure below.

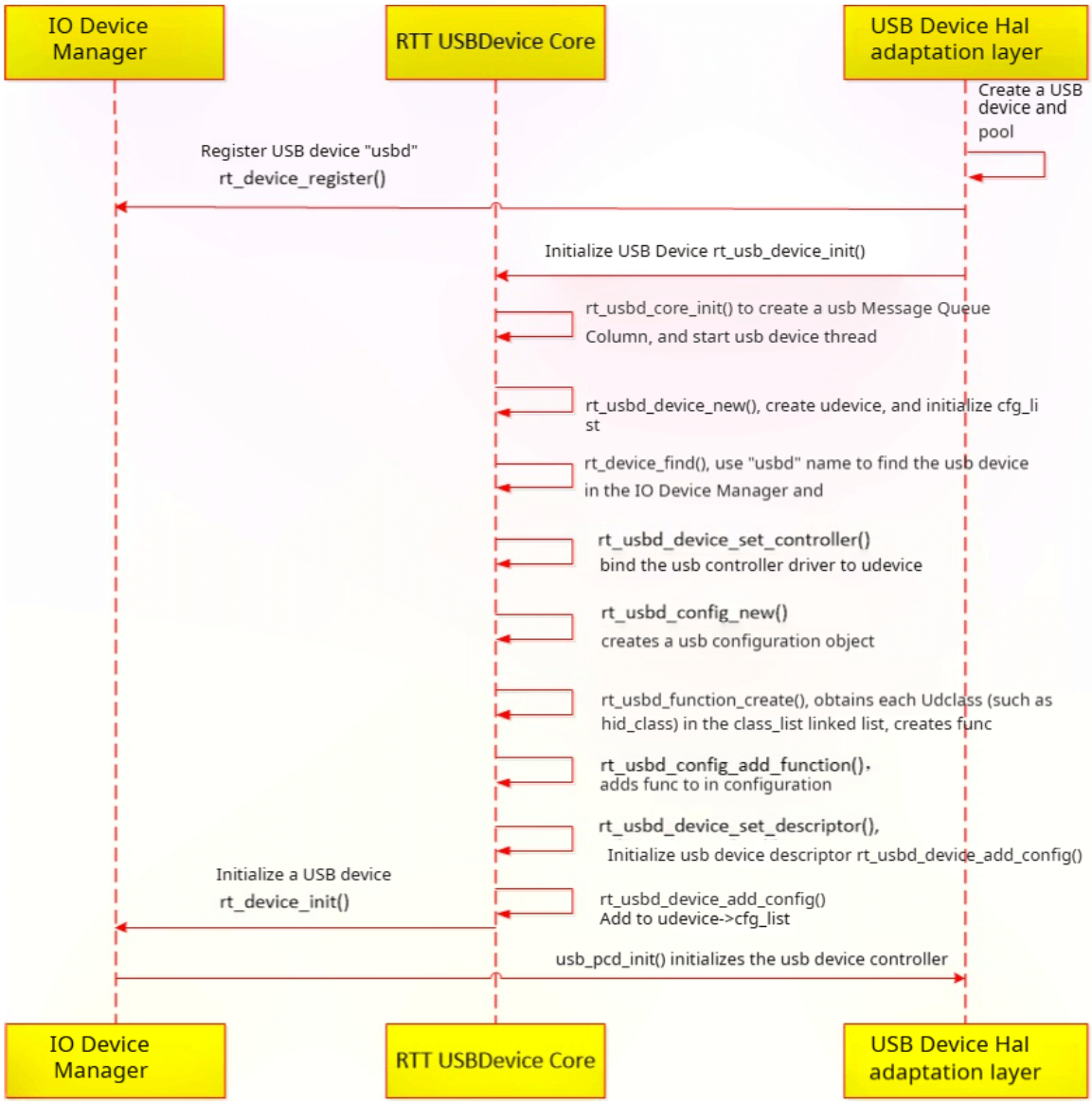


Figure 5 The creation and registration process of USB Device

Executing the command `list_device` allows you to view the generated USB Device devices:

```
msh />list_device
device           type         ref count
-------- -------------------- ----------
usbd     USB Slave Device     0
```

7.1.2 USB Device HID Usage Example

RT-Thread USB component supports common USB Device HID functions, including: Keyboard, Mouse, General HID, media keyboard. Users can configure the corresponding HID functions according to actual needs. Below is an example of RK2108 USB HID Keyboard, illustrating the configuration and usage of HID.

USB Device HID Configuration

Execute the command `scons --menuconfig` to configure USB as HID.

```
RT-Thread Components  --->
    Device Drivers  --->
        Using USB  --->
            [ ] Using USB host
            [*] Using USB device
            (4096) usb thread stack size
            (0x2207) USB Vendor ID
            (0x0007) USB Product ID
            [ ]   Enable composite device
                Device type (Enable to use device as HID device)  --->
            [*]   Use to HID device as Keyboard
            (1)     Number of Keyboard(max 3)
            [ ]   Use to HID device as Mouse
            [ ]   Use to HID device as General HID device
            [ ]   Use to HID device as media keyboard
```

Execute the command `list_device` to see the generated USB HID devices, which belong to character device type:

```
msh />list_device
device           type         ref count
-------- -------------------- ----------
hidd     Character Device     0
```

USB Device HID Creation and Registration Process

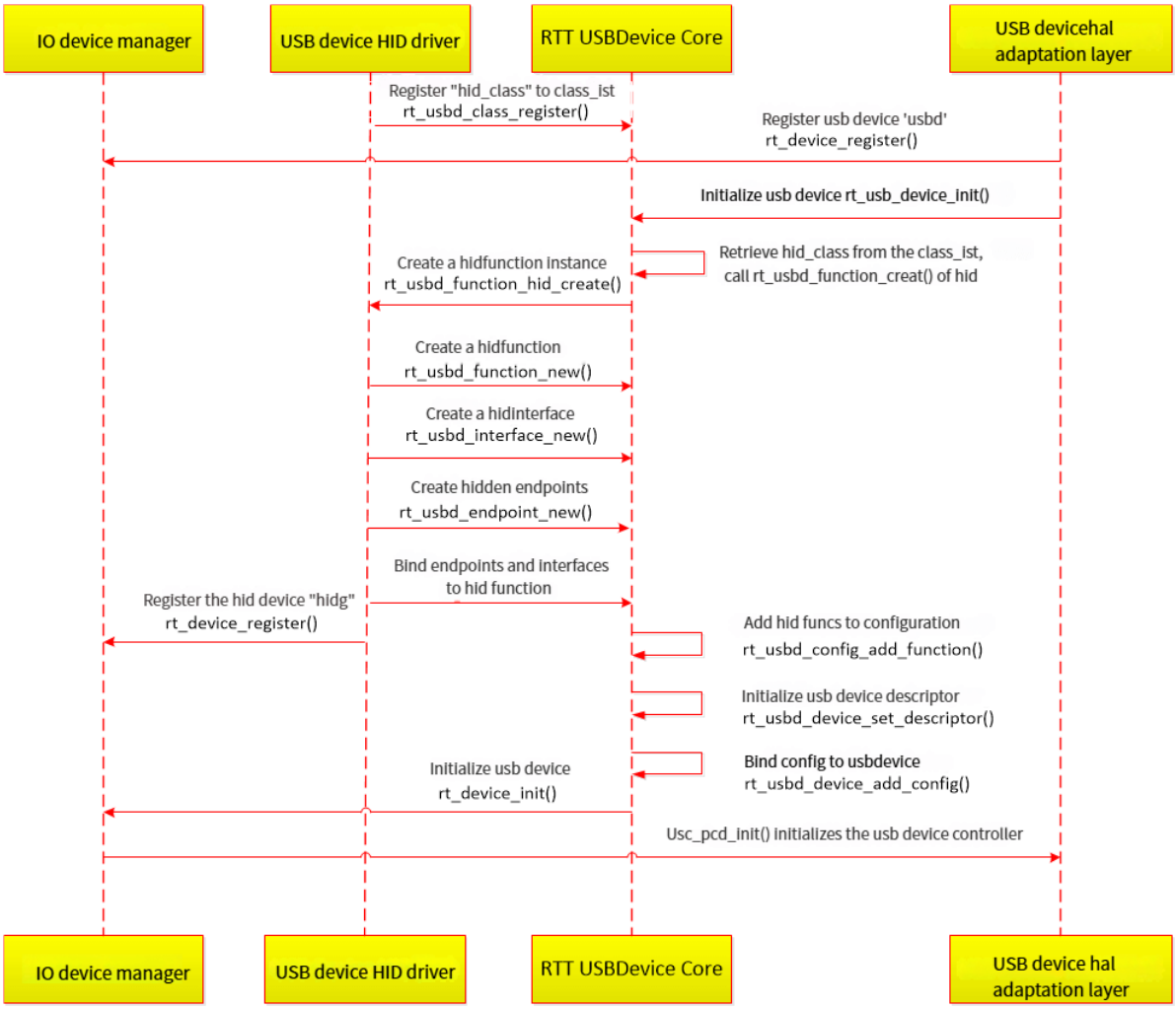


Figure 6 USB Device HID Creation and Registration Process

Application Access to HID Device Process

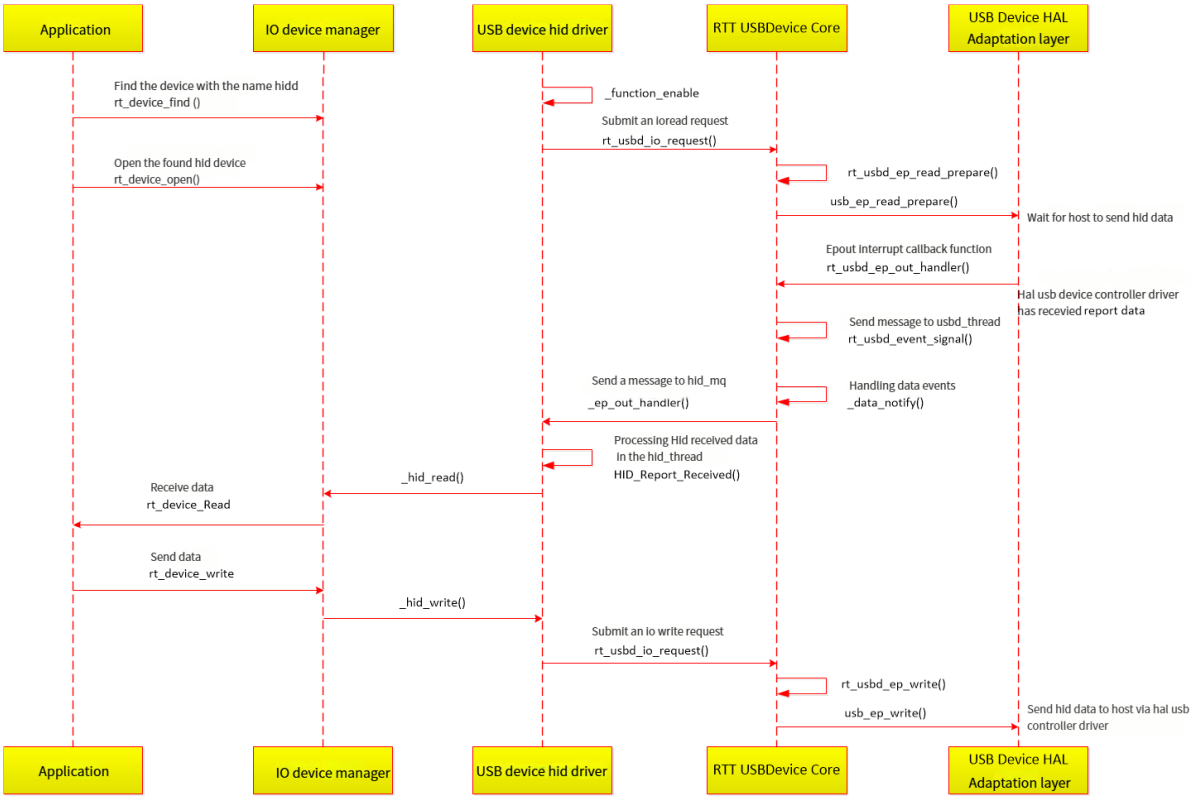


Figure 7 HID Application Access to Device Process

USB Device HID Application Development Reference Example

RT-Thread/IoT_Board's USB Mouse Example

RT-Thread RK2108 HID Reference Example

USB Device HID Transfer Performance Optimization

RT-Thread USB Device HID is limited by the HID driver's report descriptor and the USB controller INT endpoint's FIFO configuration, which can only transmit a maximum of 64 bytes of data at a time, resulting in relatively poor transfer performance. Refer to the section USB Device Endpoint FIFO Configuration Description to optimize the transfer performance of HID, supporting the transmission of 1024 bytes of data at a time.

Taking the RK2108 platform as an example:

```
diff --git a/bsp/rockchip/rk2108/board/common/board_base.c
b/bsp/rockchip/rk2108/board/common/board_base.c
index 79cc20854..a494d7528 100644
--- a/bsp/rockchip/rk2108/board/common/board_base.c
+++ b/bsp/rockchip/rk2108/board/common/board_base.c
@@ -484,8 +484,8 @@ RT_WEAK struct ep_id g_usb_ep_pool[] =
     { 0x2,  USB_EP_ATTR_BULK,       USB_DIR_OUT,    512,  ID_UNASSIGNED },
     { 0x3,  USB_EP_ATTR_ISOC,       USB_DIR_IN,     1024, ID_UNASSIGNED },
     { 0x4,  USB_EP_ATTR_ISOC,       USB_DIR_OUT,    512,  ID_UNASSIGNED },
-    { 0x5,  USB_EP_ATTR_INT,        USB_DIR_IN,     64,   ID_UNASSIGNED },
-    { 0x6,  USB_EP_ATTR_INT,        USB_DIR_OUT,    64,   ID_UNASSIGNED },
+    { 0x5,  USB_EP_ATTR_INT,        USB_DIR_IN,     1024,   ID_UNASSIGNED },
+    { 0x6,  USB_EP_ATTR_INT,        USB_DIR_OUT,    1024,   ID_UNASSIGNED },
     { 0xFF, USB_EP_ATTR_TYPE_MASK,  USB_DIR_MASK,   0,    ID_ASSIGNED   },
 };
 #endif
diff --git a/components/drivers/include/drivers/usb_common.h
b/components/drivers/include/drivers/usb_common.h
index d16b42ea7..2370cdc88 100644
--- a/components/drivers/include/drivers/usb_common.h
+++ b/components/drivers/include/drivers/usb_common.h
@@ -508,7 +508,7 @@ typedef struct uhid_descriptor* uhid_desc_t;
 struct hid_report
 {
     rt_uint8_t report_id;
-    rt_uint8_t report[63];
+    ALIGN(32) rt_uint8_t report[1024];
     rt_uint8_t size;
 };
 typedef struct hid_report* hid_report_t;
diff --git a/components/drivers/usb/usbdevice/class/hid.c
b/components/drivers/usb/usbdevice/class/hid.c
index f0e75178b..fdb12001e 100644
--- a/components/drivers/usb/usbdevice/class/hid.c
+++ b/components/drivers/usb/usbdevice/class/hid.c
@@ -194,14 +194,14 @@ const rt_uint8_t _report_desc[]=
     COLLECTION(1),      0x01,
     REPORT_ID(1),       HID_REPORT_ID_GENERAL,

-    REPORT_COUNT(1),    RT_USB_DEVICE_HID_GENERAL_IN_REPORT_LENGTH,
+    REPORT_COUNT(2),    0xff, 0x03,
     USAGE(1),           0x03,
```

```
     REPORT_SIZE(1),     0x08,
     LOGICAL_MINIMUM(1), 0x00,
     LOGICAL_MAXIMUM(1), 0xFF,
     INPUT(1),           0x02,

-    REPORT_COUNT(1),    RT_USB_DEVICE_HID_GENERAL_OUT_REPORT_LENGTH,
+    REPORT_COUNT(2),    0xff, 0x03,
     USAGE(1),           0x04,
     REPORT_SIZE(1),     0x08,
     LOGICAL_MINIMUM(1), 0x00,
@@ -347,7 +347,7 @@ const static struct uhid_comm_descriptor _hid_comm_desc =
         USB_DESC_TYPE_ENDPOINT,
         USB_DYNAMIC | USB_DIR_IN,
         USB_EP_ATTR_INT,
-        0x40,
+        1024,
         0x01,
     },

@@ -357,7 +357,7 @@ const static struct uhid_comm_descriptor _hid_comm_desc =
         USB_DESC_TYPE_ENDPOINT,
         USB_DYNAMIC | USB_DIR_OUT,
         USB_EP_ATTR_INT,
-        0x40,
+        1024,
         0x01,
     },
 };
@@ -607,7 +607,7 @@ static rt_size_t _hid_write(rt_device_t dev, rt_off_t pos,
const void *buffer, r
         rt_memcpy((void *)report.report,(void *)buffer,size);
         report.size = size;
         hiddev->ep_in->request.buffer = (void *)&report;
-        hiddev->ep_in->request.size = (size+1) > 64 ? 64 : size+1;
+        hiddev->ep_in->request.size = (size+1) > MAX_REPORT_SIZE ?
MAX_REPORT_SIZE : size+1;
         hiddev->ep_in->request.req_type = UIO_REQUEST_WRITE;
         rt_usbd_io_request(hiddev->func->device, hiddev->ep_in, &hiddev->ep_in-
>request);
         return size;
diff --git a/components/drivers/usb/usbdevice/class/hid.h
b/components/drivers/usb/usbdevice/class/hid.h
index 4d0f314a1..ea9938220 100644
--- a/components/drivers/usb/usbdevice/class/hid.h
+++ b/components/drivers/usb/usbdevice/class/hid.h
@@ -34,8 +34,8 @@ extern "C" {
 #define USB_HID_REQ_SET_IDLE        0x0a
 #define USB_HID_REQ_SET_PROTOCOL    0x0b

-#define MAX_REPORT_SIZE             64
-#define HID_RX_BUFSIZE              64
+#define MAX_REPORT_SIZE             1024
+#define HID_RX_BUFSIZE              1024

 /* HID Report Types */
```

```
#define HID_REPORT_INPUT           0x01
```

7.1.3 USB Device ADB Usage Example

RT-Thread officially provides the ADBD (Android Debug Bridge daemon) software package, which supports file transfer or execution of shell commands between PC and devices via USB or TCPIP.

The USB ADB functionality is based on the WINUSB driver for data transfer, and the ADBD application can interact with the USB controller by operating on the device node `/dev/winusb`.

For details on the ADBD software package, please refer to the RT-Thread official documentation center: RT-Thread Package Detail

Download link for the ADBD software package: RT-Thread ADBD GitHub

USB Device ADB Configuration

Configuring USB Components

The WINUSB device class driver for ADB is implemented in the ADBD software package at `packages/adbd-latest/core/adbusb.c`. Therefore, in the USB component, select to use a custom class.

It is recommended to configure the `USB Product ID` as 0x0006 to maintain consistency with the PID of the Rockchip AP chip platform's ADB functionality, avoiding the manual installation of PC ADB drivers.

```
RT-Thread Components  --->
    Device Drivers  --->
        Using USB  --->
            -*- Using USB device
            (4096) usb thread stack size
            (0x2207) USB Vendor ID
            (0x0006) USB Product ID
            [ ]   Enable composite device
                  Device type (Using custom class by register interface)  --->
                      (X) Using custom class by register interface
```

Configuring ADBD Online Packages

```
[*] Enable RT-Thread online packages  --->
    RT-Thread online packages  --->
        tools packages  --->
            [*] ADBD: Android Debug Bridge daemon implementation in RT-Thread  --
->
                [ ]   Using TCPIP transfer                  /* Enable TCP/IP
data transfer */
                [*]   Using USB transfer                    /* Enable USB data
transfer */
                (1280) Set transfer thread stack size       /* Set transfer
thread stack size */
                [*]   Enable Shell service                  /* Enable Shell
service */
```

```
                [*]   Enable File service                    /* Enable File
service */
                (2304)  Set file service thread stack size   /* Set file service
thread stack size */
                (2000)  Set file service receive timeout(ms) /* Set file receive
timeout */
                [ ]   Enable external MOD                    /* Enable external
module */
                [ ]   Enable ADB service discover            /* Enable LAN
discovery service */
                      Version (latest)  --->                 /* Select ADBD
software version */
```

Note

`Using TCPIP transfer` is used for TCPIP transfer mode and should be disabled; otherwise, ADBD will default to TCPIP transfer mode, affecting USB ADB functionality.

Configuring ADBD's Dependent POSIX

```
RT_USING_LEGACY [=y]
RT_USING_DFS [=y]
DFS_USING_POSIX [=y]
RT_USING_POSIX_FS [=y]
RT_USING_POSIX_DEVIO [=y]
RT_USING_POSIX_STDIO [=y]
RT_USING_POSIX_POLL [=y]
RT_USING_POSIX_SELECT [=y]
RT_USING_POSIX_SOCKET [=y]
```

Configuring the File System for ADB push/pull

Refer to the section USB Device MSC Usage Example for instructions on the "USB Device MSC File System Auto Mount Table" to enable the file system.

After completing the above configurations, save and exit the configuration interface.

Downloading ADBD Online Packages

Taking RK2118 SDK as an example:

```
~/rt-thread/bsp/rockchip/rk2118$ source ~/.env/env.sh
~/rt-thread/bsp/rockchip/rk2118$ pkgs --update
[Use Gitee server - auto decision based on IP location]
Cloning into '/home/wlf/rt-thread/bsp/rockchip/rk2118/packages/adbd-latest'...
remote: Enumerating objects: 42, done.
remote: Counting objects: 100% (42/42), done.
remote: Compressing objects: 100% (39/39), done.
remote: Total 42 (delta 2), reused 37 (delta 2), pack-reused 0
Receiving objects: 100% (42/42), 10.13 MiB | 1.74 MiB/s, done.
Resolving deltas: 100% (2/2), done.
==============================>  ADBD latest is downloaded successfully.

/home/wlf/rt-thread/bsp/rockchip/rk2118/packages/adbd-latest
==============================>  adbd update done
```

```
Operation completed successfully.
```

Modifying ADBD Package Source Code for Rockchip Platform Compatibility

```
diff --git a/core/adbusb.c b/core/adbusb.c
index 29455d0..034e9bb 100644
--- a/core/adbusb.c
+++ b/core/adbusb.c
@@ -232,7 +232,7 @@ static rt_err_t _ep_out_handler(ufunction_t func, rt_size_t
size)
 static rt_err_t _ep_in_handler(ufunction_t func, rt_size_t size)
 {
     winusb_device_t wd = (winusb_device_t)func->user_data;
-    wd->wrcnt -= size;
+    wd->wrcnt -= wd->ep_in->request.size;
     rt_wqueue_wakeup(&(wd->wq), (void *)POLLOUT);

     return RT_EOK;
@@ -284,13 +284,12 @@ static rt_err_t _interface_handler(ufunction_t func, ureq_t
setup)
     return RT_EOK;
 }

-#ifdef RT_USING_POSIX
 static int _ep_alloc_request(uep_t ep)
 {
     int size;

     size = EP_MAXPACKET(ep);
-    ep->buffer = rt_malloc(size);
+    ep->buffer = rt_malloc_align(RT_ALIGN(size, USB_DMA_ALIGN_SIZE),
USB_DMA_ALIGN_SIZE);
     if (!ep->buffer)
         return -1;

@@ -300,7 +299,6 @@ static int _ep_alloc_request(uep_t ep)

     return 0;
 }
-#endif

 static rt_err_t _function_enable(ufunction_t func)
 {
@@ -316,7 +314,7 @@ static rt_err_t _function_enable(ufunction_t func)
         return -1;
     if (_ep_alloc_request(wd->ep_in) != 0)
     {
-        rt_free(wd->ep_out->buffer);
+        rt_free_align(wd->ep_out->buffer);
         return -1;
     }
     _readreq(func, wd);
@@ -331,8 +329,8 @@ static rt_err_t _function_disable(ufunction_t func)
     RT_ASSERT(func != RT_NULL);
     wd = func->user_data;
```

```
-    rt_free(wd->ep_out->buffer);
-    rt_free(wd->ep_in->buffer);
+    rt_free_align(wd->ep_out->buffer);
+    rt_free_align(wd->ep_in->buffer);
     wd->ep_out->buffer = 0;
     wd->ep_in->buffer = 0;

diff --git a/services/shell_service.c b/services/shell_service.c
index 6252942..80e6cb0 100644
--- a/services/shell_service.c
+++ b/services/shell_service.c
@@ -344,6 +344,7 @@ static bool send_ready(struct shell_ext *ext, char *args)

 extern int libc_stdio_set_console(const char* device_name, int mode);
 extern int libc_stdio_get_console(void);
+extern int ioctl(int fildes, int cmd, ...);

 static int _shell_open(struct adb_service *ser, char *args)
 {
```

Execute the `list_device` and `mount` commands to check if the `winusb` node is created correctly and if the file system is mounted properly.

```
msh />list_device
device           type         ref count
-------- -------------------- ----------
winusb   Miscellaneous Device 0
usbd     USB Slave Device     0
root     Block Device         1
snor     MTD Device           0
fspi0    Character Device     0
i2c0     I2C Bus              0
pin      Pin Device           0
uart0    Character Device     2

msh />mount
filesystem  device  mountpoint
----------  ------  ----------
devfs       (NULL)  /dev
elm         root    /
```

ADB Functionality

The ADB tools and drivers on the host machine are located in the `adbd-latest/tools` directory of the ADBD packages.

Supports adb push/pull file transfers

Supports adb shell (After entering the adb shell command line, the Console will switch from the physical serial port to USB)

Command to exit adb shell: `exitas`

7.1.4 USB Device MSC Usage Example

The USB MSC (Mass Storage Class) functionality allows the device to be used as a USB flash drive. When the device is connected to a PC via USB, the PC can recognize a removable storage device, facilitating the copying of files between the device and the PC.

Taking the use of RT-Thread RK2108 MSC as an example:

USB Device MSC Configuration

Execute the command `scons --menuconfig` to configure USB as a Mass Storage device.

```
RT-Thread Components  --->
    Device Drivers  --->
        Using USB  --->
            [*] Using USB device
            (4096) usb thread stack size
            (0x2207) USB Vendor ID
            (0x0007) USB Product ID
            [ ]   Enable composite device
                    Device type (Enable to use device as Mass Storage device)  --
->
            (root) msc class disk name
            (4096) msc buffer length
```

Note

1. Manually modify `msc class disk name` to `root`
2. Enable `CONFIG_RT_USING_DFS_MNTTABLE`
3. `msc buffer length` is used to optimize the transfer performance of the USB controller, the larger the value, the better the transfer performance usually is. This value should not be less than 512 and should not exceed 65536.

USB Device MSC File System Auto Mount Table

The USB Device MSC function relies on the file system mount table. If the file system auto mount table is not configured, there may be a situation where the device can be enumerated but cannot be read or written.

Currently, the SDK supports Littlefs and FAT/EXFAT file systems. If the product requires a file system, you can enable the corresponding file system in `menuconfig` and create a file `mnt.c` in the board-level directory to configure the file system auto mount table. Below is an example of a mount table configuration:

Configure the file system

```
RT_USING_DFS [=y] #Overall file system switch
DFS_FILESYSTEMS_MAX [=2] #Maximum number of allowed mount nodes (i.e., partition
number), since devfs occupies one, this value should be >= (the number of
physical partitions that need to be mounted + 1)
DFS_FILESYSTEM_TYPES_MAX [=2] #Maximum number of allowed file systems, supports
elmfat and devfs, so it is 2
DFS_FD_MAX [=16] #Maximum number of file handles that can be operated on at the
same time
RT_USING_DFS_MNTTABLE [=y] #Enable automatic mount table
RT_USING_DFS_ELMFAT [=y] #Enable elmfat file system, compatible with fat file
system
RT_USING_DFS_DEVFS [=y] #Enable device file system, allowing access to device
drivers through file interfaces
RT_DFS_ELM_MAX_SECTOR_SIZE [=4096] #Set the sector size to 4096 to be compatible
with the erase block size of the underlying spi flash
```

Refer to RK2108 EVB: `bsp/rockchip/rk2108/board/rk2108d_evb/mnt.c`

```
#ifdef RT_USING_DFS_MNTTABLE
#include <dfs_fs.h>

#define PARTITION_ROOT          "root"

/********************* Private Variable Definition ***************************/
/** @defgroup MNT_Private_Variable Private Variable
 *  @{
 */

/**
 * @brief  Config mount table of filesystem
 * @attention The mount_table must be terminated with NULL, and the partition's
name
 * must be the same as above.
 */
const struct dfs_mount_tbl mount_table[] =
{
    {PARTITION_ROOT, "/", "elm", 0, 0},
#if defined(RT_USING_LITTLEFS) && defined(RT_USING_SPINAND)
    {"spinand0", "/nand", "lfs", 0, 0},
#elif defined(RT_USING_SDIO0) && defined(RT_SDCARD_MOUNT_POINT)
    {"sd0", RT_SDCARD_MOUNT_POINT, "elm", 0, 0},
#endif
    {0}
};
#endif
```

`dfs_mount_tbl` structure description

```
/* mount table */
struct dfs_mount_tbl
{
    const char   *device_name;    /* Device node name */
    const char   *path;           /* Mount point directory */
    const char   *filesystemtype; /* File system name (elm corresponds to fat) */
    unsigned long rwflag;         /* rw(1 for read-write, 0 for read-only, elm
does not support this flag, fill 0) */
    const void   *data;           /* File system private data (elm does not
support, fill 0) */
};
```

USB Device MSC Usage

After booting, if the device is not connected to a PC via USB, the `root` partition is mounted to the device's file system by default. At this time, the device can read and write to the `root` partition.

When the device is connected to a PC via USB, the USB driver will automatically unmount the `root` partition from the device's file system. At this time, the device cannot read or write to the `root` partition, and the PC can automatically recognize the USB flash drive and perform read and write operations on it.

When the device disconnects from the USB connection, the USB driver will automatically unmount the `root` partition and remount it to the device's file system.

7.1.5 USB Device UAC1 Usage Example

The UAC (USB Audio Class) function can be used as an extended sound card for PCs or other host computers. The main supported features are as follows:

1. Support for the UAC1.0 protocol.
2. Support for High speed (default) and Full speed. For more details, refer to the section USB Device Speed Configuration Guide.
3. Supports sample rates of 48 kHz, 44.1 kHz, 32 kHz, 16 kHz, and 8 kHz by default.
4. Supports 2 playback channels and 2 capture channels by default, with a sample depth of 16 bits.
5. Supports volume and mute control by default.
6. Supports independently enabling recording and playback functionality.

Table 5 UAC1 Main Macro Definitions

UAC1 Macro	Default Value (develop-v4.1.x)	Function Description
SAMPLE_RATE_MAX	48000	Configures the maximum sample rate for recording and playback, in Hz, up to 192000
SAMPLE_RATE_MIN	8000	Configures the minimum sample rate for recording and playback, in Hz, down to 8000
SAMPLERATE_NUM	5	Configures the total number of supported sample rates for recording and playback
P_NR_CHANNEL	2	Configures the number of playback channels, up to 8 channels
P_DEFAULT_SAMPLE_RATE	48000	Configures the default sample rate for playback
P_AUDIO_SAMPLE_BITS	16	Configures the playback sample bit depth, supports 16/24/32
P_PERIOD_SIZE_MS	10	Configures the playback buffer size; larger values result in greater playback latency
C_NR_CHANNEL	2	Configures the number of capture channels, up to 8 channels
C_DEFAULT_SAMPLE_RATE	48000	Configures the default sample rate for capture
C_AUDIO_SAMPLE_BITS	16	Configures the capture sample bit depth, supports 16/24/32

USB Device UAC1 Configuration (RT-Thread v3.1.x)

Taking the RK2108 SDK UAC1 configuration as an example:

```
RT-Thread Components  --->
    Device Drivers  --->
        Using USB  --->
```

```
            [ ] Using USB host
            [*] Using USB device
            (4096) usb thread stack size
            (0x2207) USB Vendor ID
            (0x0019) USB Product ID
            [ ] Enable composite device
                  Device type (Enable to use device as UAC1 device)  --->
            (es8388p) uac1 playback card name
            (pdmc) uac1 capture card name
            [*] Enable usb audio playback function
            [*] Enable usb audio record function
            [ ] Enable to use pll compensation for clock synchronization
```

USB Device UAC1 Configuration (RT-Thread v4.1.x)

Taking the RK2116/RK2118 SDK UAC1 configuration as an example:

```
RT-Thread Components  --->
    Device Drivers  --->
        Using USB  --->
            [ ] Using USB host
            [*] Using USB device
            (4096) usb thread stack size
            (0x2207) USB Vendor ID
            (0x0019) USB Product ID
            [ ] Enable composite device
                  Device type (Enable to use device as UAC1 device)  --->
            (rk_dmy_playback_card) uac1 playback card name
            (rk_dmy_capture_card) uac1 capture card name
            [*] Enable usb audio playback function
            [*] Enable usb audio record function
            [ ]   Enable to poll ep first transfer status
RT-Thread board config  --->
    hifi4 rtt app  --->
        [*] uac support components
    [*] hifi4 rtt codecs  --->
        [*] using rk dummy audio card
```

USB Device UAC1 + HID Composite Device Configuration (RT-Thread v4.1.x)

Taking the RK2116/RK2118 SDK UAC1 + HID configuration as an example:

```
RT-Thread Components  --->
    Device Drivers  --->
        Using USB  --->
            [ ] Using USB host
            [*] Using USB device
            (4096) usb thread stack size
            (0x2207) USB Vendor ID
            (0x0029) USB Product ID
            [*] Enable composite device
            [*] Enable to use device as HID device
            [*] Enable to use device as UAC1 device
            [*] Use to HID device as media keyboard
            (rk_dmy_playback_card) uac1 playback card name
```

```
            (rk_dmy_capture_card) uac1 capture card name
            [*] Enable usb audio playback function
            [*] Enable usb audio record function
            [*] Enable to poll ep first transfer status

RT-Thread board config  --->
    hifi4 rtt app  --->
        [*] uac support components
    [*] hifi4 rtt codecs  --->
        [*] using rk dummy audio card
```

Note

1. When UAC1 functionality is enabled, VAD functionality must be disabled, i.e., set `# CONFIG_RT_USING_VAD is not set`.
2. `RT_USB_AUDIO_P_NAME` corresponds to the actual playback device node and should be modified according to the actual audio hardware design. The RK2108 SDK reference design is `es8388p`, and the RK2116/RK2118 SDK reference design is `rk_dmy_playback_card`.
3. `RT_USB_AUDIO_C_NAME` corresponds to the actual recording device node and should be modified according to the actual audio hardware design. The RK2108 SDK reference design is `pdmc`, and the RK2116/RK2118 SDK reference design is `rk_dmy_playback_card`.
4. `RT_USB_AUDIO_PLL_COMPENSATION` is used to enable PLL compensation for dynamic clock adjustment on the device side. This feature is used in device playback scenarios and can solve the problem of buffer overrun or underrun caused by the desynchronization between the USB transmission clock (from USB HOST SOF) and the I2S clock of the UAC1 device. After enabling this feature, the device side can actively adjust its own data processing clock to adapt to the data transmission rate of the USB HOST. The advantage of this method is that it does not depend on USB host systems such as Windows/Linux/MAC OS and has better compatibility. The disadvantage is that the UAC1 device side requires software development for dynamic clock adjustment according to different SoC clock design schemes, which may be limited by the chip's clock design. It is currently mainly applied on the RK2108 chip platform.
5. `RT_USB_AUDIO_USING_EP_POLL` indicates the use of the HAL layer USB audio endpoint polling method to replace the default interrupt method for obtaining the status of audio stream start transmission. If the product supports UAC1 composite devices (e.g., UAC1 + HID), it is recommended to enable it by default to avoid the potential risk of endpoint interrupt storms (related to the behavior of the connected USB host), but at the same time, the polling method will also introduce the impact of audio stream latency. The SDK default setting for the polling interval `EP_STATUS_POLL_INTERVAL_MS` is 5ms. If the product is sensitive to latency, the polling interval can be modified according to application requirements.
6. The UAC1 driver in RT-Thread v3.1.x does not support USB audio endpoint polling by default and can be supported by updating the driver patch.

   Patch download address: https://redmine.rock-chips.com/attachments/1217535

After the system starts, if the playback device is not found, the PC can recognize the UAC1 device but cannot play audio. If the recording device is not found, the system will fault. Therefore, before enabling the UAC1 function, please use the `list_device` command to check if the Sound device has been created. Taking the RK2108 SDK as an example, `es8311p` corresponds to the audio playback device, and `pdmc` corresponds to the audio recording device.

If no audio playback and recording devices are found, please open the corresponding Audio compilation switches in menuconfig.

```
msh />list_device
device          type         ref count
------- -------------------- ----------
es8311c Sound Device         0
es8311p Sound Device         1
audpwmp Sound Device         0
pdmc    Sound Device         0
adcc    Sound Device         0
usbd    USB Slave Device     0
root    Block Device         1
snor    MTD Device           0
pwm0    Miscellaneous Device 0
dmac0   Miscellaneous Device 0
i2c0    I2C Bus              0
pin     Miscellaneous Device 0
uart2   Character Device     0
uart0   Character Device     2
```

7.1.6 USB Device UAC2 Usage Example

The UAC (USB Audio Class) function can be used as an extended sound card for PCs or other host computers. The main supported features are as follows:

1. Support for the UAC2.0 protocol.
2. Support for High speed (default) and Full speed. For more details, refer to the section USB Device Speed Configuration Guide.
3. Supports playback sample rates of 192 kHz, 96 kHz, 48 kHz, and 44.1 kHz by default.
4. Supports capture sample rates of 96 kHz, 48 kHz, and 44.1 kHz by default.
5. Supports 2 playback channels and 2 capture channels by default, with a sample depth of 16 bits.
6. Supports volume and mute control by default.
7. Supports independently enabling recording and playback functionality.
8. Does not support high-bandwidth transmission of 3 KB/microframe.

Table 6 UAC2 Main Macro Definitions

UAC2 Macro	Default Value (develop-v4.1.x)	Function Description
P_NR_CHANNEL	2	Configures the number of playback channels, up to 16 channels
P_AUDIO_SAMPLE_BITS	16	Configures the playback sample bit depth, supports 16/24/32
P_PERIOD_SIZE_MS	5	Configures the playback buffer size; larger values result in greater playback latency
P_SAMPLE_RATE_DEFAULT	48000	Configures the default sample rate for playback
C_NR_CHANNEL	2	Configures the number of capture channels, up to 16 channels
C_AUDIO_SAMPLE_BITS	16	Configures the capture sample bit depth, supports 16/24/32
C_SAMPLE_RATE_DEFAULT	48000	Configures the default sample rate for capture
UAC2_HS_INTERVAL_DEFAULT	0x04	Configures the HS USB audio frame interval, default 1ms (2^(4-1) = 8 uframes)
UAC2_FS_INTERVAL_DEFAULT	0x01	Configures the FS USB audio frame interval, fixed at 1ms

USB Device UAC2 Configuration (RT-Thread v3.1.x)

Taking the RK2108 SDK UAC2 configuration as an example:

```
RT-Thread Components  --->
    Device Drivers  --->
```

```
        Using USB  --->
            [ ] Using USB host
            [*] Using USB device
            (4096) usb thread stack size
            (0x2207) USB Vendor ID
            (0x0019) USB Product ID
            [ ] Enable composite device
                  Device type (Enable to use device as UAC2 device)  --->
            (es8388p) uac playback card name
            (pdmc) uac capture card name
            [*] Enable usb audio playback function
            [*] Enable usb audio record function
```

USB Device UAC2 Configuration (RT-Thread v4.1.x)

Taking the RK2116/RK2118 SDK UAC2 configuration as an example:

```
RT-Thread Components  --->
    Device Drivers  --->
        Using USB  --->
            [ ] Using USB host
            [*] Using USB device
            (4096) usb thread stack size
            (0x2207) USB Vendor ID
            (0x0019) USB Product ID
            [ ] Enable composite device
                  Device type (Enable to use device as UAC2 device)  --->
            (rk_dmy_playback_card) uac playback card name
            (rk_dmy_capture_card) uac capture card name
            [*] Enable usb audio playback function
            [*] Enable usb audio record function
            [ ]   Enable to poll ep first transfer status
RT-Thread board config  --->
    hifi4 rtt app  --->
        [*] uac support components
    [*] hifi4 rtt codecs  --->
        [*] using rk dummy audio card
```

USB Device UAC1 + HID Composite Device Configuration (RT-Thread v4.1.x)

7.1.7 USB Device UVC Usage Example

UVC (USB Video Class) functionality allows devices to be used as USB video capture devices. Rockchip RT-Thread UVC is implemented based on the UVC 1.1 standard protocol, supporting MJPEG and YUYV formats of video data, and currently supports up to three UVC composite devices.

UVC Usage Restrictions:

1. Only BULK transfer mode is supported, ISOC transfer mode is not supported;
2. Only supported on the RK625 chip platform.

USB Device UVC Configuration

```
RT-Thread Components  --->
    Device Drivers  --->
        Using USB  --->
            [*] Using USB device
            (4096) usb thread stack size
            (0x2207) USB Vendor ID
            (0x0007) USB Product ID
            [*]   Enable composite device
            [*]     Enable to use device as UVC device
            [*]   Use to UVC device as RGB
            [ ]   Use to UVC device as DEPTH
            [ ]   Use to UVC device as IR
            (10240) uvc device max buffer length
```

Note

1. RGB, DEPTH, IR need to be configured according to the specific hardware support of the SoC. For example: RK625 SoC can support RGB UVC, RK625 + RK1608 solution can support three UVC routes;
2. `uvc device max buffer length` can support configuration in the range of 4K-16K.

UVC Design Overview

The UVC component is based on the RT-Thread USB Framework and mainly includes UVC threads, UVC drivers, Buffer management, and exception handling sub-modules. The system block diagram is shown below.

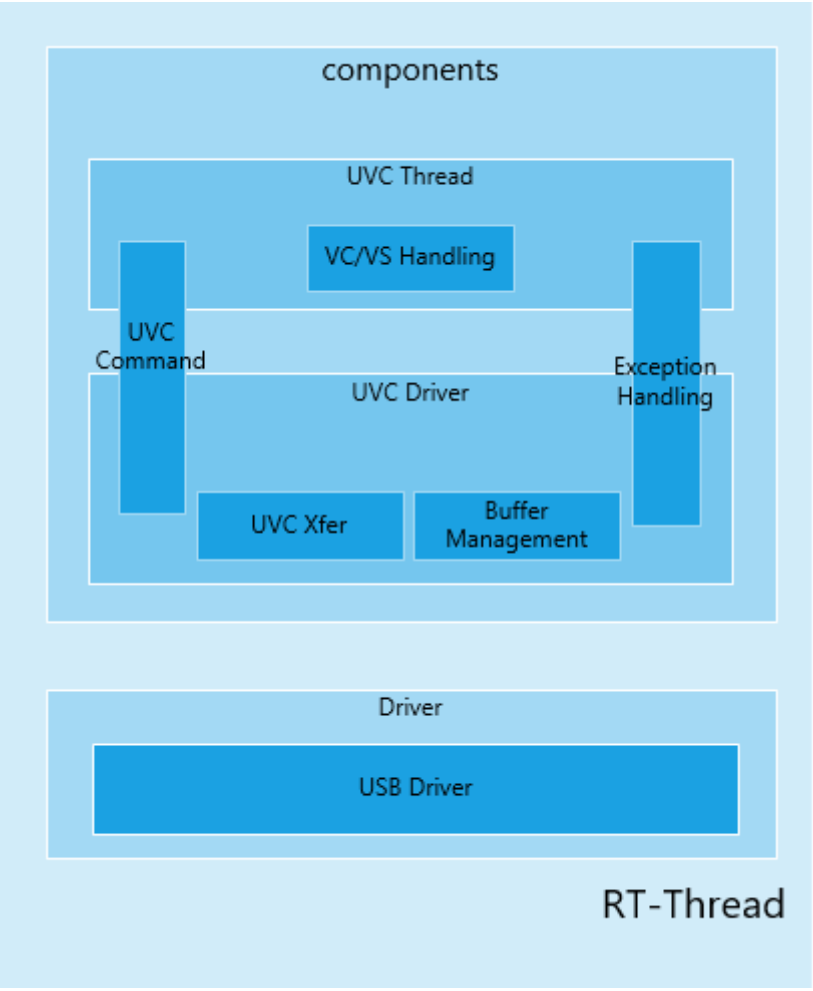


Figure 8 UVC Component Block Diagram

- UVC Thread mainly implements UVC VC/VS request response, with a separate processing thread for each UVC;
- UVC Driver implements UVC device descriptor definition (including image resolution definition), UVC Buffer status management, UVC video data transfer scheduling, and exception handling functions;
- UVC Thread and UVC Driver communicate through RT-Thread OS Event; UVC Driver and USB Driver communicate through USB IO interface.

UVC Log Analysis

Connect the UVC device to the host computer, and use video preview software on the host computer to open the Camera. After the Rockchip SoC serial port is powered on and the Camera is initialized, the following UVC-related Log output will be available:

```
[...1...]
INF: uvc(1) set format fcc 1196444237 width 640 height 480
VIDEO_OPEN(1)
INF: uvc(1) open camera device...
Open camera id width height 1 640 480
[...2...]
INF: uvc(1) buffer(0) addr 0x2006787c
INF: uvc(1) buffer(1) addr 0x2006987c
INF: uvc(1) buffer(2) addr 0x2006b87c
INF: uvc(1) buffer(3) addr 0x2006d87c
INF: uvc(1) mjpeg header addr 0x20087c00
INF: uvc(1) buffer size 8064
[...]
[...3...]
VIDEO_CLOSE(1)
INF: uvc(1) close camera device...
Close camera id 1
close rgb success
[...]
[...4...]
uvc(1) buffer: (0-1) (1-1) (2-1) (3-2)
ERR: uvc(1) buffer overflow irq 0x000100f0
```

- Log 1 indicates the FCC and width/height information of the opened Camera;
- Log 2 indicates the number of UVC ring Buffers used, their addresses, and the size of each Buffer; if it is MJPEG format data, MJPEG Header Buffer information will also be printed;
- After Log 1 and Log 2 are successfully output, if the Sensor has data to send, UVC will schedule Buffer management and transfer data to the host computer;
- Log 3 indicates the Log when the host computer closes the Camera;
- Log 4 indicates that common video preview software such as AMCAP, GUVCView, etc., do not send a Close Camera command when closing the preview or exiting, but only send an EP Halt command to USB. After EP Halt, it will trigger UVC Buffer Overflow, and the Buffer manager will then close the Buffer channel, stopping USB data transfer.

7.1.8 USB Device WINUSB Usage Example

WINUSB is a generic USB driver provided by the Windows operating system, enabling Bulk data transfer without the need for driver installation.

USB Device WINUSB Configuration

```
RT-Thread Components  --->
    Device Drivers  --->
        Using USB  --->
            [ ] Using USB host
            [*] Using USB device
            (4096) usb thread stack size
            (0x2207) USB Vendor ID
            (0x0007) USB Product ID
            [ ]   Enable composite device
                    Device type (Enable to use device as winusb device)  --->
            ({6860DC3C-C05F-4807-8807-1CA861CC1D66}) Guid for winusb
            (16384) winusb transfer length
```

Executing the command `list_device` allows you to view the generated USB WINUSB devices:

```
msh />list_device
device           type         ref count
-------- -------------------- ----------
winusb   Miscellaneous Device 0
usbd     USB Slave Device     0
```

For WINUSB functionality, Rockchip provides a host-side Demo that can be used for WINUSB functionality testing.

Download Path: https://redmine.rock-chips.com/attachments/1967928

7.1.9 USB Device CDC_VCOM Usage Example

CDC_VCOM supports USB as a virtual serial port for data transmission with the host computer, based on the CDC-ACM protocol for driver implementation.

USB Device CDC_VCOM Configuration

```
RT-Thread Components  --->
    Device Drivers  --->
        Using USB  --->
            [ ] Using USB host
            [*] Using USB device
            (4096) usb thread stack size
            (0x2207) USB Vendor ID
            (0x0007) USB Product ID
            [ ]   Enable composite device
                    Device type (Enable to use device as CDC device)  --->
            (512) virtual com thread stack size
            (128) virtual com rx buffer size
```

```
            [ ]   Enable to use dma for vcom tx
            (32021919830108) serial number of virtual com
            (14)  serial number length of virtual com
            (1000) tx timeout(ticks) of virtual com
```

RT-Thread 3.1.x Method for Implementing USB System Console

```
diff --git a/components/libc/compilers/newlib/libc.c
b/components/libc/compilers/newlib/libc.c
index 2234c36f98..07ba2bafeb 100644
--- a/components/libc/compilers/newlib/libc.c
+++ b/components/libc/compilers/newlib/libc.c
@@ -23,6 +23,7 @@ int   _EXFUN(putenv,(char *__string));

 int libc_system_init(void)
 {
+    rt_console_set_device("vcom");
 #if defined(RT_USING_DFS) & defined(RT_USING_DFS_DEVFS) &
defined(RT_USING_CONSOLE)
     rt_device_t dev_console;

@@ -47,4 +48,4 @@ int libc_system_init(void)

     return 0;
 }
-INIT_COMPONENT_EXPORT(libc_system_init);
+INIT_APP_EXPORT(libc_system_init);
```

RT-Thread 4.1.x Method for Implementing USB System Console

Support dynamic console switching:

- When USB is connected to the host and the host opens the USB serial device using a serial port tool (e.g., Microcom/Minicom/SecureCRT), the console automatically switches to USB VCOM.
- When USB is disconnected from the host, the console automatically switches back to the default console configured at system startup.

Dependency on POSIX Configuration

```
RT_USING_POSIX_FS [=y]
RT_USING_POSIX_DEVIO [=y]
RT_USING_POSIX_STDIO [=y]
```

```
diff --git a/components/drivers/usb/usbdevice/class/cdc_vcom.c
b/components/drivers/usb/usbdevice/class/cdc_vcom.c
index ba70a04b27..9de44a5041 100644
--- a/components/drivers/usb/usbdevice/class/cdc_vcom.c
+++ b/components/drivers/usb/usbdevice/class/cdc_vcom.c
@@ -82,6 +82,7 @@ struct vcom
     rt_uint8_t tx_rbp[CDC_TX_BUFSIZE];
     struct rt_ringbuffer tx_ringbuffer;
     struct rt_event  tx_event;
+    rt_device_t old_ser_device;
```

```
 };

 struct vcom_tx_msg
@@ -464,8 +465,18 @@ static rt_err_t _interface_handler(ufunction_t func, ureq_t
setup)
         break;
     case CDC_SET_CONTROL_LINE_STATE:
         data->connected = (setup->wValue & 0x01) > 0?RT_TRUE:RT_FALSE;
-        RT_DEBUG_LOG(RT_DEBUG_USB, ("vcom state:%d \n", data->connected));
+        rt_kprintf("usb vcom connect state: %d \n", data->connected);
         dcd_ep0_send_status(func->device->dcd);
+        if (data->connected)
+        {
+            if (data->old_ser_device)
+            {
+                rt_kprintf("switch old console %s to usb vcom.\n",
+                           data->old_ser_device->parent.name);
+            }
+            rt_console_set_device("vcom");
+            finsh_set_device("vcom");
+        }
         break;
     case CDC_SEND_BREAK:
         break;
@@ -525,6 +536,12 @@ static rt_err_t _function_disable(ufunction_t func)
     _vcom_reset_state(func);

     data = (struct vcom*)func->user_data;
+    if (data->old_ser_device)
+    {
+        rt_console_set_device(data->old_ser_device->parent.name);
+        finsh_set_device(data->old_ser_device->parent.name);
+    }
+
     if(data->ep_out->buffer != RT_NULL)
     {
         rt_free_align(data->ep_out->buffer);
@@ -958,6 +975,8 @@ static void rt_usb_vcom_init(struct ufunction *func)
     data->serial.serial_rx  = RT_NULL;
     data->serial.config     = config;

+    data->old_ser_device = rt_console_get_device();
+
     /* register vcom device */
     rt_hw_serial_register(&data->serial, VCOM_DEVICE,
 #ifndef VCOM_TX_USE_DMA
@@ -987,4 +1006,41 @@ int rt_usbd_vcom_class_register(void)
 }
 INIT_PREV_EXPORT(rt_usbd_vcom_class_register);

+int usb_vcom_enable(void)
+{
+    rt_device_t cdc_vcom;
+    rt_device_t old_device;
+
```

```
+    cdc_vcom = rt_device_find("vcom");
+    if (cdc_vcom == RT_NULL)
+    {
+        rt_kprintf("not found usb vcom device!\n");
+        return -RT_ERROR;
+    }
+
+    old_device = rt_console_get_device();
+    if (old_device)
+    {
+        rt_kprintf("switch old console %s to usb vcom.\n",
+                   old_device->parent.name);
+    }
+
+    rt_console_set_device(cdc_vcom->parent.name);
+    finsh_set_device(cdc_vcom->parent.name);
+
+    return RT_EOK;
+}
+MSH_CMD_EXPORT(usb_vcom_enable, enable usb cdc virtual console);
+
+int usb_vcom_disable(void)
+{
+    /* Switch to default console device */
+    rt_kprintf("switch usb vcom to %s.\n", RT_CONSOLE_DEVICE_NAME);
+    rt_console_set_device(RT_CONSOLE_DEVICE_NAME);
+    finsh_set_device(RT_CONSOLE_DEVICE_NAME);
+
+    return RT_EOK;
+}
+MSH_CMD_EXPORT(usb_vcom_disable, disable usb cdc virtual console);
+
 #endif
```

CDC_VCOM Compatibility Issue Resolution

Problem Description: Communicating with the host computer through the USB virtual serial port, the Rockchip Device can receive serial messages from the host, but the Device cannot send serial messages to the host normally.

Solution:

```
Subject: [PATCH] [COMPONENTS]: drivers: usb: cdc_vcom: Fix connected state

The current cdc vcom driver depends the setup wValue of
the request CDC_SET_CONTROL_LINE_STATE to set the connected
state. Refer to CDC Spec V1.2 Chapter 6.3.12 SetControlLineState,
the Bit0 of the wValue is used to indicates to DCE if DTE
(Data Terminal Equipment, for example, a Personal Computer)
is present or not. However, some DTE always set the Bit0 of
the wValue to 0, this cause cdc vcom failed to send message to
DTE. This patch ignores the Bit0 of wValue, and set the
connected state to true directly if it receives the request
CDC_SET_CONTROL_LINE_STATE.
```

```
diff --git a/components/drivers/usb/usbdevice/class/cdc_vcom.c
b/components/drivers/usb/usbdevice/class/cdc_vcom.c
index df5e03f54c..e201742747 100644
--- a/components/drivers/usb/usbdevice/class/cdc_vcom.c
+++ b/components/drivers/usb/usbdevice/class/cdc_vcom.c
@@ -463,7 +463,7 @@ static rt_err_t _interface_handler(ufunction_t func, ureq_t
setup)
         _cdc_get_line_coding(func->device, setup);
         break;
     case CDC_SET_CONTROL_LINE_STATE:
-        data->connected = (setup->wValue & 0x01) > 0?RT_TRUE:RT_FALSE;
+        data->connected = RT_TRUE;
         RT_DEBUG_LOG(RT_DEBUG_USB, ("vcom state:%d \n", data->connected));
         dcd_ep0_send_status(func->device->dcd);
         break;
```

7.2 Example of Using RT-Thread 5.1.x USB Device

7.2.1 USB Device ADB Usage Example

RT-Thread 5.1.x ADB is based on the ADBD (Android Debug Bridge daemon) software package provided by RT-Thread official and has been improved to support USB3.0 transfer, while also optimizing the transfer rate of PUSH/PULL. The source code is integrated into the `rt-thread/third_party/adbd` directory, eliminating the need to download ADBD online packages.

Taking the RK1820 USB ADB usage as an example:

```
RT-Thread Components  --->
    Device Drivers  --->
        [*] Using USB with CherryUSB  --->
            [*]   Enable usb device mode  --->
                  Select usb device speed (AUTO) /* Select USB Speed automatic
detection */
                  Select usb device ip (drd_rk)  /* Select USB Device Controller
IP */
                  Select usb device template (none (Implement it yourself))

RT-Thread third party package  --->
     [*] ADBD: Android Debug Bridge daemon implementation in RT-Thread  --->
         [ ]   Using TCPIP transfer
         [*]   Using USB transfer
         (4096) Set transfer thread stack size
         [*]   Enable Shell service
         [*]   Enable File service
         (4096)  Set file service thread stack size
         (5000)  Set file service receive timeout(ms)
         [ ]   Enable external MOD
         [ ]   Enable ADB service discovery

RT-Thread rockchip RK1820 drivers  --->
     Enable USB DRD Device  --->
         [*] Enable USB DRD Device controllers
```

```
          [*]   Enable USB DRD_0 Device
```

Note

The ADB driver requires a relatively large buffer size, it is recommended that the Memory size in the board config options be at least 2MB.

Executing the command `list_device` will show the generated USB ADB device

```
msh />list device
device           type         ref count
-------- -------------------- ----------
usb_adb  Miscellaneous Device 0
```

ADB Features

The ADB tools and drivers for the host computer are located in the `adbd-latest/tools` directory of the ADBD packages.

Support for adb push/pull files

Support for adb shell (After entering the adb shell command line, the Console will switch from the physical serial port to USB)

Command to exit adb shell: `exitas`

7.2.2 USB Device WINUSB Usage Example

CherryUSB Device protocol stack currently supports WINUSB1.0, WINUSB2.0, and WEBUSB functionalities. Taking the use of RK1820 WINUSB as an example:

```
RT-Thread Components  --->
    Device Drivers  --->
        [*] Using USB with CherryUSB  --->
            [*]   Enable usb device mode  --->
                  Select usb device speed (AUTO) /* Select USB Speed automatic
detection */
                  Select usb device ip (drd_rk)  /* Select USB Device controller
IP */
                  Select usb device template (winusbv1_rt) /* Select WINUSB1.0
RTT driver */
                      (winusb0) usb device winusb interface0 name
                      [ ]       Enable usb device with double winusb /* Select
support for two independent winusb interfaces */

RT-Thread rockchip RK1820 drivers  --->
     Enable USB DRD Device  --->
         [*] Enable USB DRD Device controllers
         [*]   Enable USB DRD_0 Device
```

Executing the command `list_device` allows you to see the generated USB WINUSB device:

```
msh />list_device
device           type         ref count
-------- -------------------- ----------
winusb0  Miscellaneous Device 0
```

WINUSB Test Case Reference

```
/**
  * Copyright (c) 2025 Fuzhou Rockchip Electronics Co., Ltd
  *
  * SPDX-License-Identifier: Apache-2.0
  */
#include <rtthread.h>
#include <rtdevice.h>
#include <unistd.h>
#include <fcntl.h>
#include <errno.h>
#include <sys/stat.h>
#include <stdio.h>
#include <math.h>
#include "drv_heap.h"

typedef struct utc_d_client_s
{
    rt_device_t device;
    rt_uint8_t *rev_buffer;
    rt_uint8_t *snd_buffer;
    rt_uint32_t package_len;
    rt_sem_t rx_notice;
    rt_sem_t tx_notice;
    rt_uint8_t send_busy_flag;
} utc_d_client_t;

static utc_d_client_t utc_d_client;

static rt_err_t utc_d_rx_ind(rt_device_t dev, rt_size_t size)
{
    utc_d_client_t *client = &utc_d_client;

    /*read the usb data next */
    rt_device_read(dev, 0, client->rev_buffer, 512);
    if (size)
    {
        rt_kprintf("utc_d_rx_ind rev size = %d \n", size);
    }
    return RT_EOK;
}

static rt_err_t utc_d_tx_ind(rt_device_t dev, void *buffer)
{
    utc_d_client_t *client = &utc_d_client;

    rt_kprintf("utc_d_tx_ind snd_buffer = %p \n", buffer);
    client->send_busy_flag = 0;
```

```
    return RT_EOK;
}

static void winusb_thread(void *p)
{
    utc_d_client_t *client = &utc_d_client;

    while (1)
    {
        if (!client->send_busy_flag)
        {
            client->send_busy_flag = 1;
            rt_device_write(client->device, 0, client->snd_buffer, 512);
        }
        rt_thread_mdelay(10);
    }
}

int winusb_app_init(void)
{
    utc_d_client_t *client = &utc_d_client;
    client->device = rt_device_find("winusb");
    rt_err_t result;

    if (client->device)
    {
        rt_kprintf("find winusb device\n");
        /* using the tx interrupt when uart is RS485 */
        result = rt_device_open(client->device, RT_DEVICE_OFLAG_RDWR);
        RT_ASSERT(result == RT_EOK);
        client->rx_notice = rt_sem_create("winusb_rx", 0, RT_IPC_FLAG_FIFO);
        client->tx_notice = rt_sem_create("winusb_tx", 0, RT_IPC_FLAG_FIFO);

        client->rev_buffer = (rt_uint8_t *)rt_malloc_align(512, 64);
        client->snd_buffer = (rt_uint8_t *)rt_malloc_align(512, 64);
        rt_memset(client->rev_buffer, 0, 512);
        rt_memset(client->snd_buffer, 0x31, 512);

        client->send_busy_flag = 0;

        rt_device_set_tx_complete(client->device, utc_d_tx_ind);
        rt_device_set_rx_indicate(client->device, utc_d_rx_ind);

        /*wait read the winusb device success */
        while (rt_device_read(client->device, 0, client->rev_buffer, 512) == 0)
        {
            rt_thread_mdelay(500);
        }
    }
    else
    {
        rt_kprintf("Not find winus device\n");
        return RT_ERROR;
    }
```

```
    rt_thread_t thread;

    thread = rt_thread_create("clockdark", winusb_thread, RT_NULL, 2048, 5, 10);
    RT_ASSERT(thread != RT_NULL);

    rt_thread_startup(thread);
    return RT_EOK;
}

INIT_APP_EXPORT(winusb_app_init);
```

For the WINUSB functionality, Rockchip provides a host-side Demo that can be used for WINUSB functionality testing.

Download Path: Demo Download

7.2.3 USB Device CDC_ACM Usage Example

CherryUSB Device protocol stack currently supports Communication Device Class (CDC_ACM), using RK1820 CDC_ACM as an example:

```
RT-Thread Components  --->
    Device Drivers  --->
        [*] Using USB with CherryUSB  --->
            [*]   Enable usb device mode  --->
                  Select usb device speed (AUTO) /* Select USB Speed Auto
Detection */
                  Select usb device ip (drd_rk)  /* Select USB Device Controller
IP */
                  [*]   Enable usb cdc acm device /* Select CDC ACM Class Driver
*/
                  Select usb device template (cdc_acm) /* Select CDC ACM Device
Driver */

RT-Thread rockchip RK1820 drivers  --->
     Enable USB DRD Device  --->
         [*] Enable USB DRD Device controllers
         [*]   Enable USB DRD_0 Device
```

Note

The CDC_ACM DEMO driver provided by the SDK `components/drivers/usb/cherryusb/demo/cdc_acm_template.c` does not create and register the corresponding ACM device in the RT-Thread I/O Device Manager. If the product requires accessing the ACM device through an application, it is necessary to create an ACM device node using the `rt_device_register` interface. Reference can be made to the code implementation of the SDK WINUSB&CDC_ACM composite device driver in `components/drivers/usb/cherryusb/demo/winusb1.0_cdc_acm_rt.c`.

7.2.4 USB Device CDC_VCOM Usage Example

CDC_VCOM supports USB acting as a virtual serial port for data transmission and reception with the host computer. The driver is implemented based on the CDC-ACM protocol. It can output console information via USB and perform console command interaction. Take the RK1820 CDC_VCOM as an example:

```
RT-Thread Components  --->
    Device Drivers  --->
        [*] Using USB with CherryUSB  --->
            [*]   Enable usb device mode  --->
                  Select usb device speed (AUTO) /* Select USB Speed Auto
Detection */
                  Select usb device ip (drd_rk)  /* Select USB Device Controller
IP */
                  -*-   Enable usb cdc acm device /* Select CDC ACM Class Driver
*/
                  -*-     Enable usb cdc acm as virtual com /* Select CDC VCOM
Class Driver */
                  (1024) virtual com thread stack size
                  (128) virtual com rx buffer size
                  [ ]   Enable to use dma for vcom tx
                  (1000) tx timeout(ticks) of virtual com
                      Select usb device template -->
                          (X) cdc_acm_vcom /* Select CDC VCOM Device Driver */
RT-Thread rockchip RK1820 drivers  --->
     Enable USB DRD Device  --->
         [*] Enable USB DRD Device controllers
         [*]   Enable USB DRD_0 Device
```

7.2.4.1 USB Virtual Console Switching Methods

Method 1.

- When USB is connected to the host and the host opens the USB serial device using a serial port tool (e.g., Microcom/Minicom/SecureCRT), the console automatically switches to USB VCOM.
- When USB is disconnected from the host, the console automatically switches back to the default console configured at system startup.

Method 2.

- On the default console at system startup (typically `RT_CONSOLE_DEVICE_NAME`), switch to USB VCOM using the following console command:
  `msh /> usb_vcom_enable`
- On the USB VCOM console, switch back to the default console at system startup using the following command:
  `msh /> usb_vcom_disable`

7.2.5 USB Device WINUSB&CDC_ACM Composite Device Usage Example

Support 1 x WINUSB Interface + 1 x CDC_ACM Interface + 1 x CDC_VCOM Interface

```
RT-Thread Components  --->
    Device Drivers  --->
        [*] Using USB with CherryUSB  --->
            [*]   Enable usb device mode  --->
                  Select usb device speed (AUTO) /* Select USB Speed Auto
Detection */
                  Select usb device ip (drd_rk)  /* Select USB Device Controller
IP */
                  -*-   Enable usb cdc acm device /* Select CDC ACM Class Driver
*/
                  [*]     Enable usb cdc acm as virtual com/* Select CDC VCOM
Class Driver */
                  (1024) virtual com thread stack size
                  (128) virtual com rx buffer size
                  [ ]   Enable to use dma for vcom tx
                  (1000) tx timeout(ticks) of virtual com
                      Select usb device template -->
                          (X) (X) winusbv1_cdc_acm_rt /* Select WINUSB&CDC_ACM
Device Driver */
RT-Thread rockchip RK1820 drivers  --->
     Enable USB DRD Device  --->
         [*] Enable USB DRD Device controllers
         [*]   Enable USB DRD_0 Device
```

Execute the command `list device` to see the generated USB WINUSB&CDC_ACM composite device:

```
device           type         ref count
-------- -------------------- ----------
acm      Character Device     0
winusb0  Character Device     0
usb_vcom Character Device     0
```

Table 7 WINUSB&CDC_ACM Interface Function Description

Interface	Interface Name	Function Description
WINUSB	NTB Interface	Used for transmitting RK1820 NPU data
CDC_ACM	SMI Interface[Note1]	Used for transmitting RK1820 SMI information
CDC_VCOM	VCOM Interface[Note2]	Used for transmitting RK1820 console information and console command interaction

Note1. On the Linux USB host side, the corresponding SMI Interface can be found by querying `/sys/class/tty/ttyACM*/device/interface`.

Note2. On the Linux USB host side, the corresponding VCOM Interface can be found by querying `/sys/class/tty/ttyACM*/device/interface`. For instructions on using the USB Virtual Console, refer to the section USB Virtual Console Switching Methods.

8. USB Host Usage Example

8.1 Example of Using RT-Thread 3.1.x/4.1.x USB Host

8.1.1 USB Host Creation and Registration

USB Host is registered into the RT-Thread I/O device manager through the `rt_device_register()` interface. A USB Hub thread is created to monitor the ports in real-time, enumerating devices when the port status changes. Additionally, by initializing a device class driver chain list and adding different drivers to the list, management of various device class drivers is achieved.

RT-Thread supports three different types of USB HOST controller driver adaptation layers, namely: DWC2/EHCI/OHCI. For corresponding code paths, please refer to the section USB Code Path.

Taking the creation and registration of the DWC2 controller as an example, the process is shown in Figure 9 below.

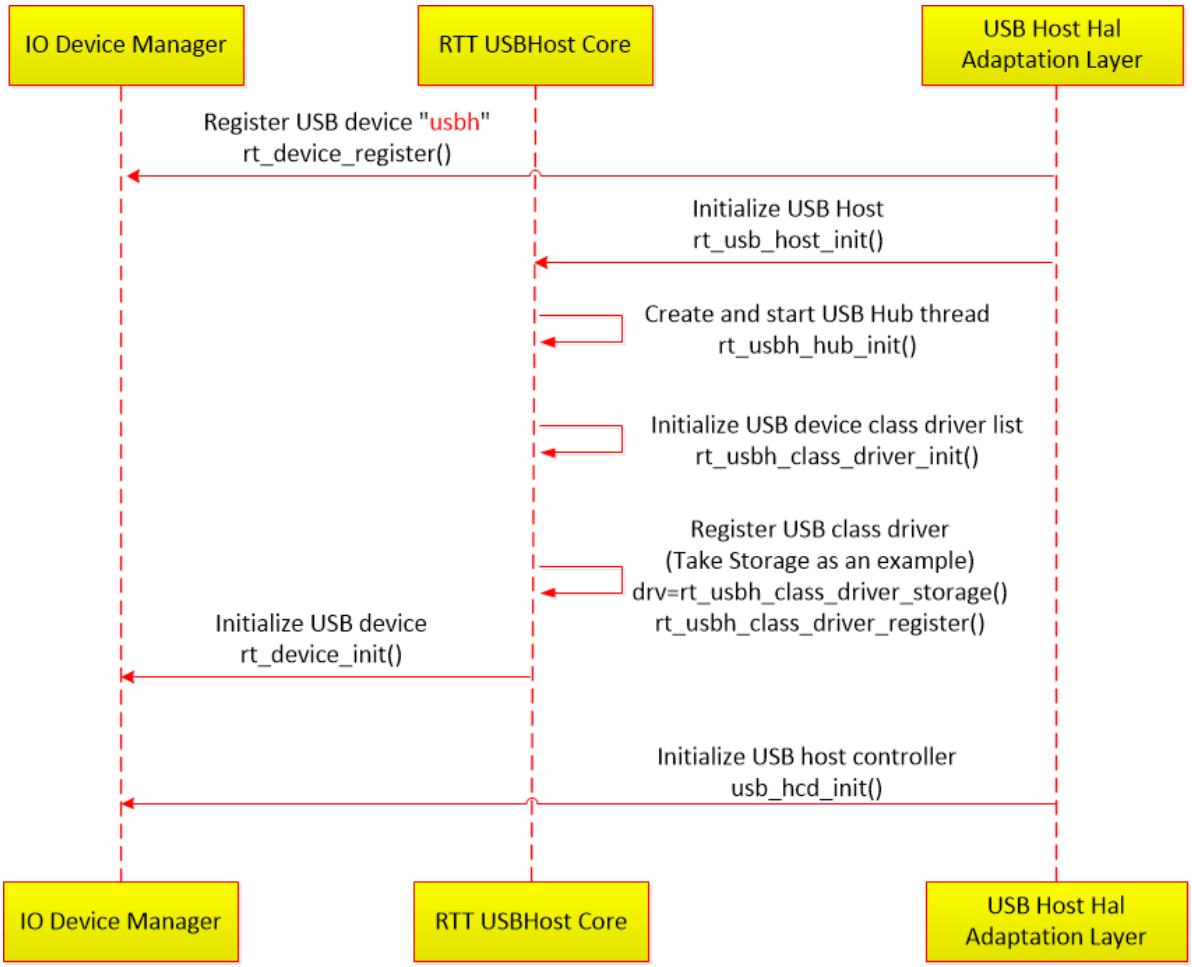


Figure 9 USB Host Creation and Registration Process

Note

If the controller type is EHCI/OHCI, the corresponding driver adaptation layer file is `bsp/rockchip/common/drivers/drv_usbhost.c`. The creation and registration process is essentially the same as for the DWC2 controller.

Executing the command `list_device` allows you to see the generated USB Host devices:

```
msh />list_device
device           type         ref count
-------- -------------------- ----------
usbh     USB Host Bus         0
```

8.1.2 USB Host MSC Usage Example

USB Host MSC Component Configuration

```
RT-Thread Components  --->
    Device Drivers  --->
        Using USB  --->
        [*] Using USB host
        [*]   Enable Udisk Drivers   /* Select Udisk driver */
        (/udisk)     Udisk mount dir /* Set the Udisk mount directory, can be
modified */
        [ ]   Enable HID Drivers
        [ ] Using USB device
        (4096) usb thread stack size /* Set the USB host thread stack size */
```

USB Host Driver Adaptation Layer Configuration

RT-Thread 3.1.x USB Host Driver Adaptation Layer Configuration

Configuration should be based on the actual USB controller model supported by the chip platform and the RT-Thread software version.

```
RT-Thread rockchip common drivers  --->
    RT-Thread rockchip USB Host driver  --->
        [ ] DWC2 HCD Support
        [ ] EHCI HCD Support
        [ ] OHCI HCD Support
```

Note

1. `DWC2 HCD Support` corresponds to the DWC2 controller driver adaptation layer;
2. `EHCI HCD Support` and `OHCI HCD Support` correspond to the USB2 Host EHCI & OHCI controller driver adaptation layers;

RT-Thread 4.1.x USB Host Driver Adaptation Layer Configuration

To support the simultaneous operation of multiple USB Host interfaces on different chip platforms, starting from RT-Thread v4.1.x SDK, the configuration of the USB Host driver adaptation layer has been moved from the platform common driver to the chip driver.

Taking the RK3506 chip platform as an example, which supports two Host interfaces OTG0/1 simultaneously, the configuration is as follows:

```
RT-Thread rockchip RK3506 drivers  --->
    Enable USB HOST  --->
        [*] Enable USB OTG HOST
        [*]   Enable USB OTG0 HOST
        [*]   Enable USB OTG1 HOST
```

USB Host MSC Function Dependency File System Related Configuration

```
RT-Thread Components  --->
    [*] DFS: device virtual file system  ----
        [*] Enable elm-chan fatfs
              elm-chan's FatFs, Generic FAT Filesystem Module  --->
                ...
                (4) Number of volumes (logical drives) to be used. /* The total
number of supported Drives/Volumes */
                (4096) Maximum sector size to be handled. /* The maximum sector
size supported */
```

Note

1. By default, only FAT-32 formatted Udisks are supported. To support exFAT format, you need to add `#define RT_DFS_ELM_USE_EXFAT`.
2. The configuration item `Number of volumes (logical drives) to be used` includes the total number of supported Drives/Volumes, such as partitions mounted by the Udisk and partitions mounted through the mount table (e.g., root partition and data partition). During product development, this parameter should be correctly configured according to the actual number of supported partitions.
3. Udisk automatic mounting is supported, but only single-partition mounting is allowed. If the Udisk has multiple partitions, only the first partition will be mounted.
4. Udisk Application supports unmounting the Udisk through the standard interface `rt_device_control()`. For specific methods, refer to Table 4.

USB Host MSC Function Dependency Cache Configuration

```
RT-Thread rockchip common drivers  --->
    [*] Enable cache
```

USB Host MSC Device Management Interface

Table 8 RT-Thread 4.1.x Udisk Device Management Interface

Function	Description
rt_udisk_read()	Read data from Udisk Support for POSIX file operation interface `read` and C standard library function `fread`
rt_udisk_write()	Write data to Udisk Support for POSIX file operation interface `write` and C standard library function `fwrite`

Function	Description
rt_udisk_control()	Control Udisk unmount RT_USBH_CLASS_CTRL_ENABLE: Enable the permission to control Udisk unmount RT_USBH_CLASS_CTRL_CONNECTED: Query the connection status of the Udisk RT_USBH_CLASS_CTRL_STOP: Perform the operation to unmount the Udisk

USB Host MSC Registration and Enumeration Process

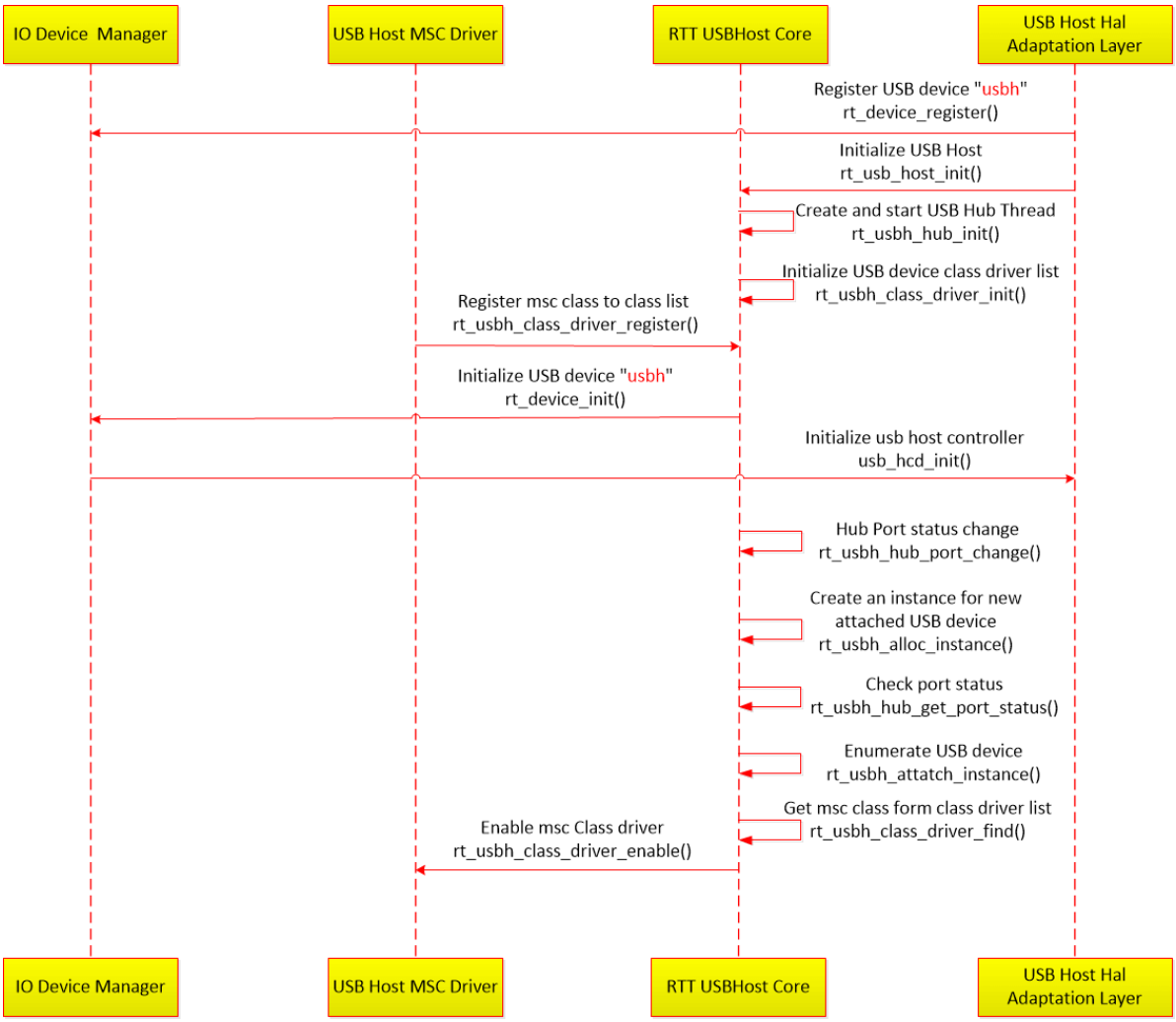


Figure 10 USB Host MSC Registration and Enumeration Process

MSC Device Access Process in Application

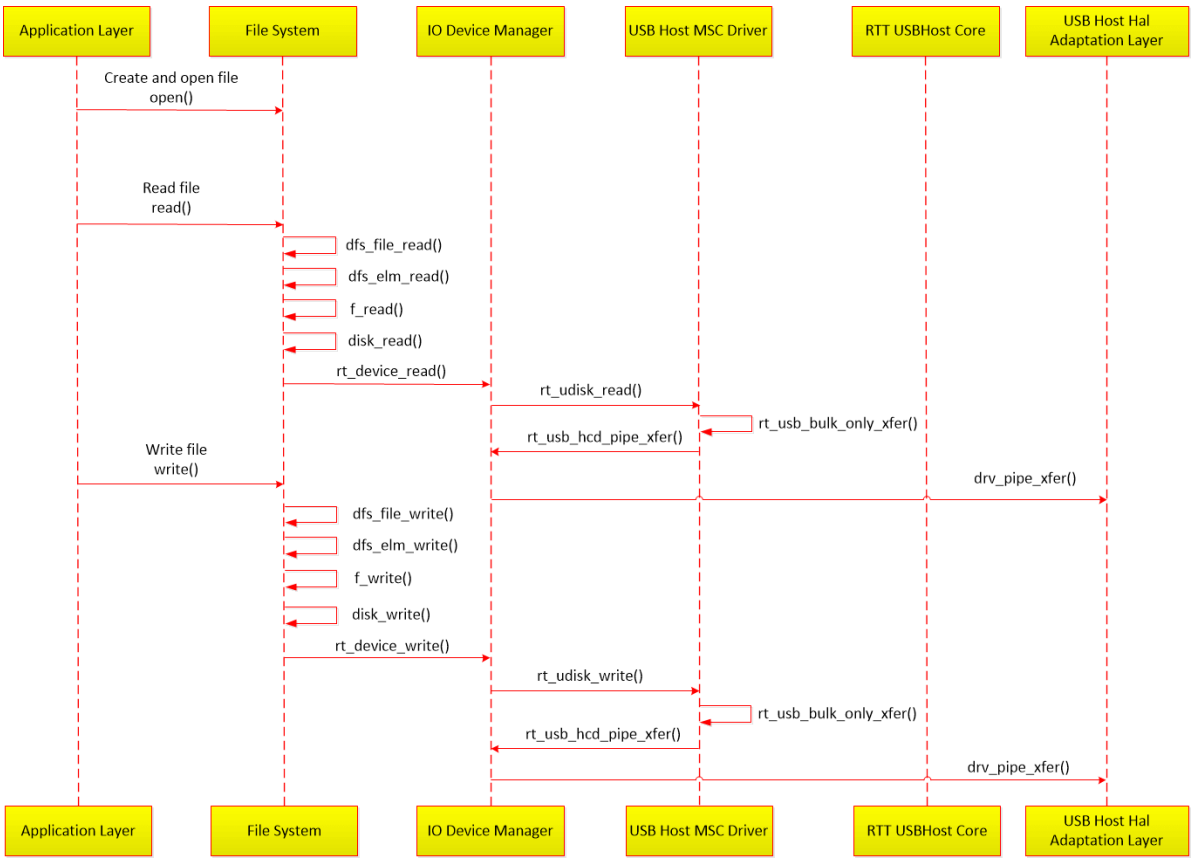


Figure 11 MSC Application Device Access Process

8.2 Example of Using RT-Thread 5.1.x USB Host

RT-thread 5.1.x CherryUSB Host currently only supports the xHCI controller, and it supports Control transfer and Bulk transfer in terms of transfer modes.

8.2.1 USB Host MSC Usage Example

The CherryUSB Host MSC driver supports automatic mounting and read/write operations for Udisk.

Taking the RK1820 USB1 Host MSC as an example:

```
RT-Thread Components  --->
    DFS: device virtual file system  --->
        [*] Enable elm-chan fatfs
              elm-chan's FatFs, Generic FAT Filesystem Module  --->
                ...
                (4) Number of volumes (logical drives) to be used. /* The total
number of supported Drives/Volumes */
                (4096) Maximum sector size to be handled. /* The maximum sector
size supported */

    Device Drivers  --->
        [*] Using USB with CherryUSB  --->
            [*] Enable usb host mode  --->
                  Select usb host ip (xhci_rk)    /* Select USB Host Controller
IP xhci_rk */
```

```
                ...
                [*] Enable usb msc driver       /* Select Udisk driver */
                (/udisk) Udisk mount dir        /* Set the Udisk mount
directory, can be modified */

RT-Thread rockchip RK1820 drivers  --->
    Enable USB XHCI HOST  --->
        [*] Enable USB XHCI HOST controllers
        [*] Enable USB XHCI1 HOST
```

Note

1. By default, only FAT-32 formatted Udisks are supported. To support exFAT format, you need to add `#define RT_DFS_ELM_USE_EXFAT`.
2. The configuration item `Number of volumes (logical drives) to be used` includes the total number of supported Drives/Volumes, such as partitions mounted by the Udisk and partitions mounted through the mount table (e.g., root partition and data partition). During product development, this parameter should be correctly configured according to the actual number of supported partitions.
3. Udisk automatic mounting is supported, but only single-partition mounting is allowed. If the Udisk has multiple partitions, only the first partition will be mounted.
4. Udisk Application supports unmounting the Udisk through the standard interface `rt_device_control()`. For specific methods, refer to Table 5.

USB Host MSC Device Management Interface

Table 9 RT-Thread 5.1.x Udisk Device Management Interface

Function	Description
rt_udisk_read()	Read data from Udisk Support for POSIX file operation interface `read` and C standard library function `fread`
rt_udisk_write()	Write data to Udisk Support for POSIX file operation interface `write` and C standard library function `fwrite`
rt_udisk_control()	Control Udisk unmount RT_CHERRY_USBH_CTRL_ENABLE: Enable the permission to control Udisk unmount RT_CHERRY_USBH_CTRL_CONNECTED: Query the connection status of the Udisk RT_CHERRY_USBH_CTRL_STOP: Perform the operation to unmount the Udisk